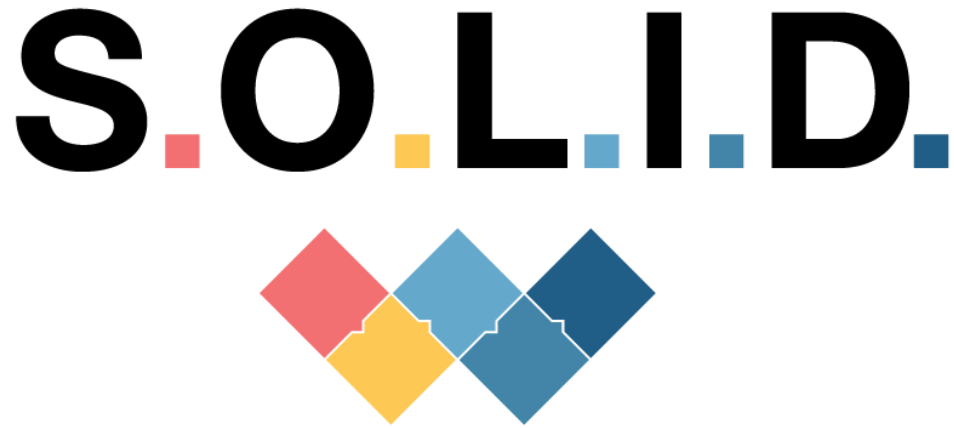
SOLID Design Principles



COP 4934

SOLID

- Single Responsibility Principle
- Open Closed Principle
- Liskov's Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

What is Software Design

- Real software runs on computers. It is a sequence of ones and zeros that is stored on some magnetic media. It is not a program listing in C++ (or any other programming language).
- A program listing is a document that represents a software design. Compilers and linkers actually build software designs.
- Real software is incredibly cheap to build, and getting cheaper all the time as computers get faster.
- Real software is incredibly expensive to design. This is true because software is incredibly complex and because practically all the steps of a software project are part of the design process.

Programs are ones and zeros, not a source code listing.

Source code is a document representing software design.

Software is cheap to build.

Software is expensive to design.

More About Software Design

- Programming is a design activity—a good software design proces
recognizes this and does not hesitate to code when coding make
- Coding actually makes sense more often than believed. Often the process of rendering the design in code will reveal oversights and the need for additional design effort. The earlier this occurs, the better the design will be.
- Since software is so cheap to build, formal engineering vali are not of much use in real world software development. It cheaper to just build the design and test it than to try to prove it.
- Testing and debugging are design activities—they are the software equivalent of the design validation and refinement processes of other engineering disciplines. A good software design process recognizes this and does not try to short change the steps.

Design Activity

Rendering reveals oversights.

Sometimes Software is Built without Formal engineering.

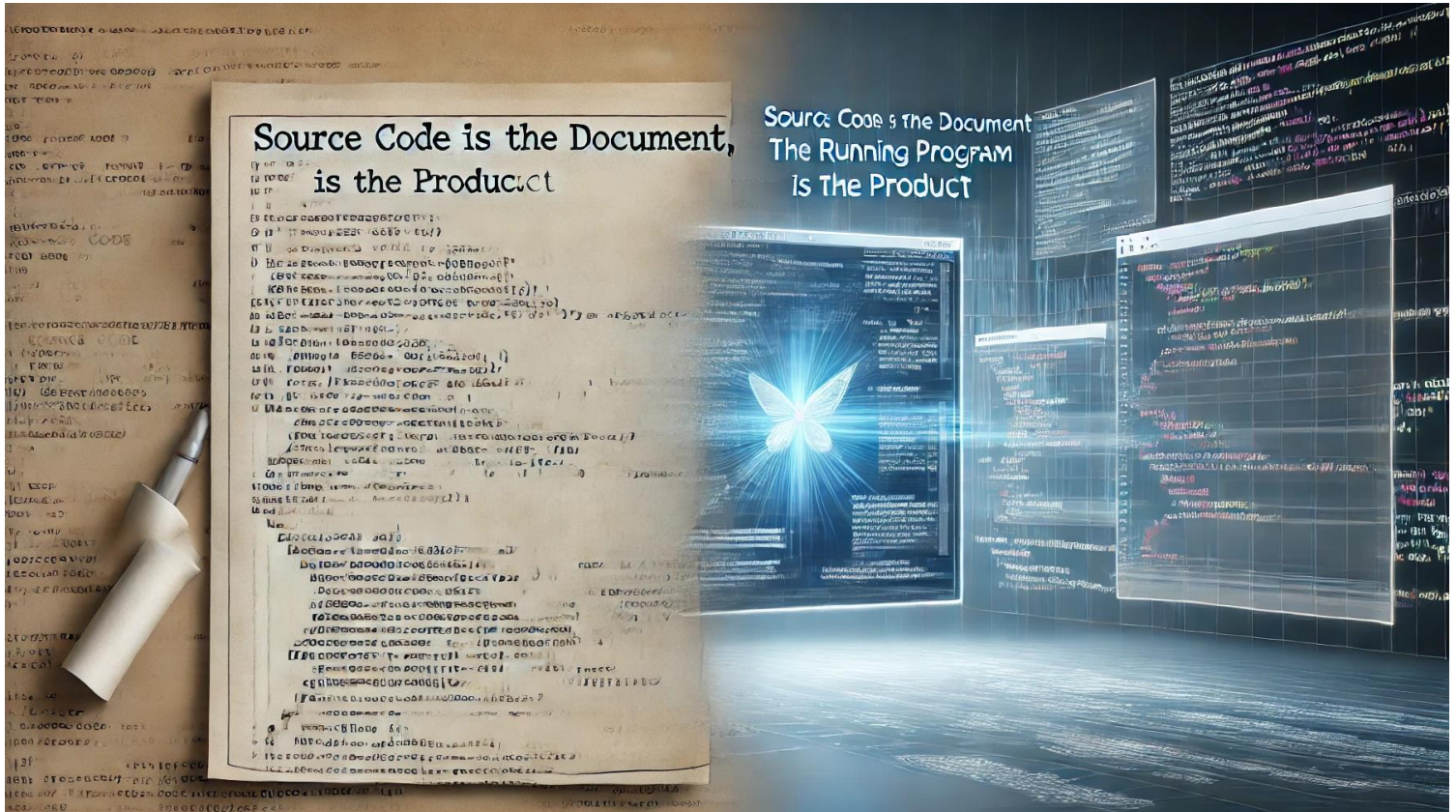
Testing and debugging are design activities.

Even More About Software Design

- There are other design activities—call them top level design, module design, structural design, architectural design, or whatever. A good software design process recognizes this and deliberately includes the steps.
- All design activities interact. A good software design process recognizes this and allows the design to change, sometimes radically, as various design steps reveal the need.
- Many different software design notations are potentially useful—as auxiliary documentation and as tools to help facilitate the design process. They are not a software design.
- Software development is still more a craft than an engineering discipline. This is primarily because of a lack of rigor in the critical processes of validating and improving a design.
- Ultimately, real advances in software development depend upon advances in programming techniques, which in turn mean advances in programming languages. C++ is such an advance. It has exploded in popularity because it is a mainstream programming language that directly supports better software design.
- C++ is a step in the right direction, but still more advances are needed.

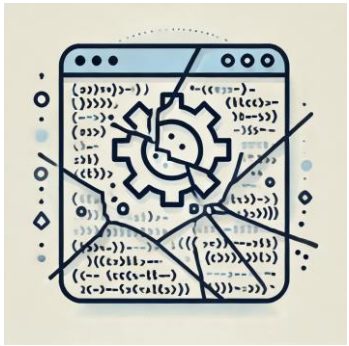
Engineers Produce Documents

- Source code is the document, running program is the product



Symptons of bad design

- Rigidity
- Fragility
- Immobility
- Viscosity
- Needless Complexity



Rigidity

- Tendency of a system to be hard to change.
- The cost of making a change is high (imagine jenga).

- First: takes a long time to rebuild and test.

- Second: Only a small change requires a rebuild and test.
- What kind of structure supports easy changes?
- Rigidity can be a sign of high coupling.
- You can manage dependencies.

Fragility

- A small change to one module causes other unrelated modules to misbehave.
- Management and customers often see this as a sign of significant incompetence.
- Manage dependencies between modules and isolate them from each other.

Immobility

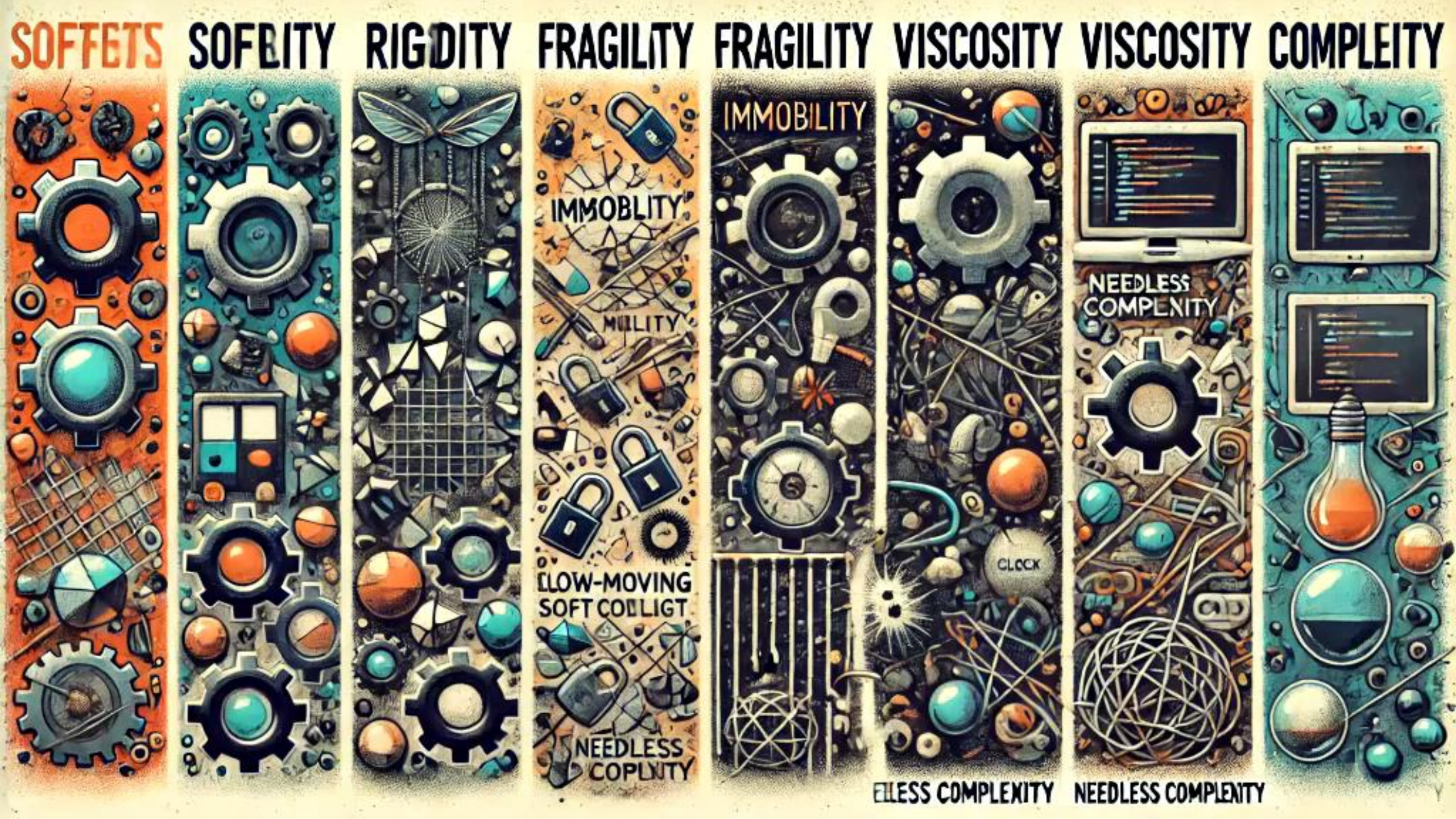
- When its internal components cannot be easily extracted and used in other environments
- Example: Login module
- Avoid: decouple central abstractions from DB, UI, and frameworks

Viscosity

- Necessary operations such as building and testing difficult to perform take a long time
- Check ins/Check Outs/Merges take a long time
- New features must be added over multiple layers of system
- Cause: Irresponsible tolerance - do not correct bad behaviors - tight coupling
- Cure: Attack systems - decouple modules and manage dependencies

Needless Complexity

- Deal with future. Design for today only or anticipate future.
- Carry lots of anticipation is needlessly complex.
- Can represent that you are afraid of changing your code in the future and litter it with all sorts of hooks.
- Without this designs will be simpler.
- This can lead to tight coupling.
- Solution is to keep your focus on current set of requirements with enough tests to make sure changes don't break things.


SOFTETS
SOFLITY
RIGDITY
FRAGILITY
IMMOBLITY
MULITY
LOW-MOVING
SOFT COLLIGT
NEEDLESS
COPLNTY
FRAGILITY
IMMOBILITY
VISCOSITY
CLOCK
ELESS COMPLEXITY
VISCOSITY
NEEDLESS
COMPLNITY
NEEDLESS COMPLEXITY
COMPLEITY

Code Rot

- Example: Create a function that allows you to type characters on the keyboard and then outputs them to the printer.

```
void copy()
{
        int c;
        while( ( c = readKeyboard() != EOF )
        {
                writePrinter(c);
        }
}
```

Ammendment

- Everyone is using the copy() function. But we need to get input from a paper tape. (You will get a lot of angry developers if you change the copy() function.)

```
bool readFlag = false;
// Remember to clear

void copy()
{
        int c;
        while( ( c = (readFlag ? readPt() : readKeyboard()) != EOF )
        {
                writePrinter(c);
        }
}
```

Another Ammendment

- Function is widely used. Need top optionally output to punch tape.

```
bool readFlag = false;
bool writeFlag = false;
// Remember to clear

void copy()
{
        int c;
        while( ( c = (readFlag ? readPt() : readKeyboard()) != EOF )
        {
                writeFlag ? writePunch() : writePrinter(c);
        }
}
```

Best Approach

getchar() and putchar() can talk to any device.

```
void copy()
{
        int c;
        while( ( c == getchar() ) != EOF )
        {
                putchar(c);
        }
}
```

SOLID

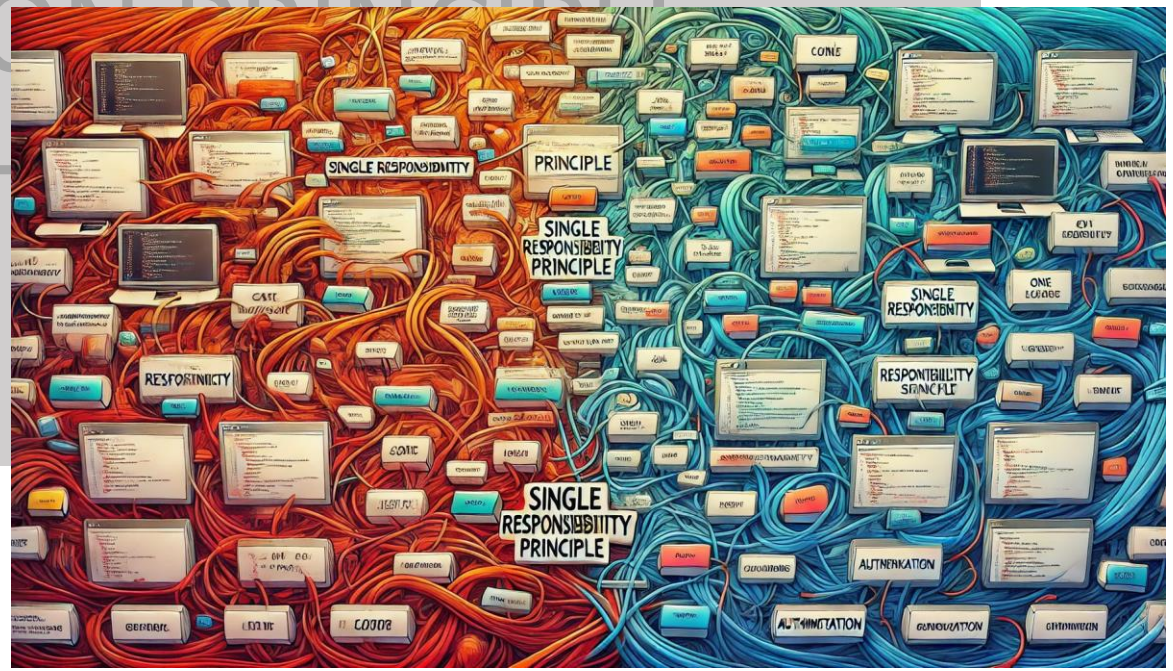
SINGLE RESPONSIBILITY PRINCIPLE

OPEN CLOSED PRINCIPLE

LISKOV'S SUBSTITUTION PRINCIPLE

INTERFACE SEGREGAT

DEPENDENCY INVERS



Single Responsibility Principle

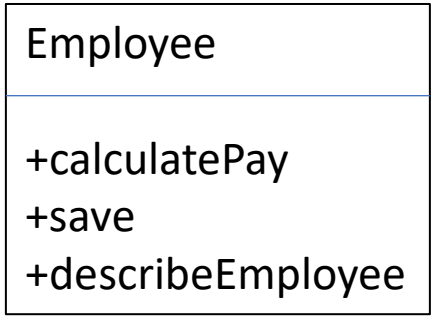
- This is about functions and modules.
- The best ones have just one responsibility.
- But what is responsibility?

Functions and Modules

One Responsibility

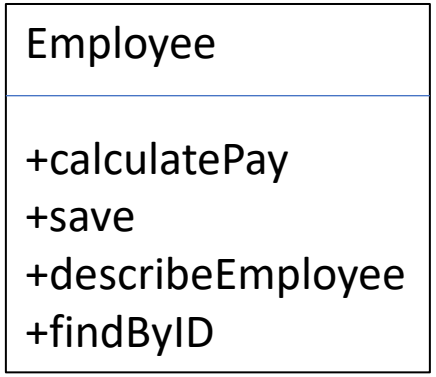
What does that mean?

Class Employee (part of a payroll application)



It seems that there are three responsibilities.

Class Employee

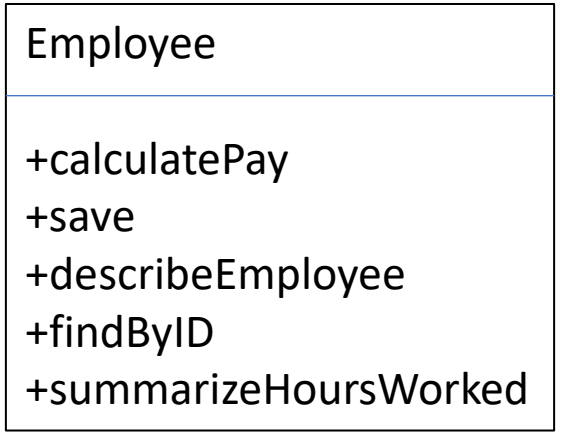


Now I have added a new method that finds an employee given an ID.

There are still three responsibilities since the new method is in the same family as save. They are both database functions.

If we added calculateDeductions() and calculateTaxes() these could be considered in the same family as calculatePay()

Class Employee



summarizeHoursWorked returns a string with a summary. Which family does it belong to. Ask are the users who will use this method.

Responsibilities

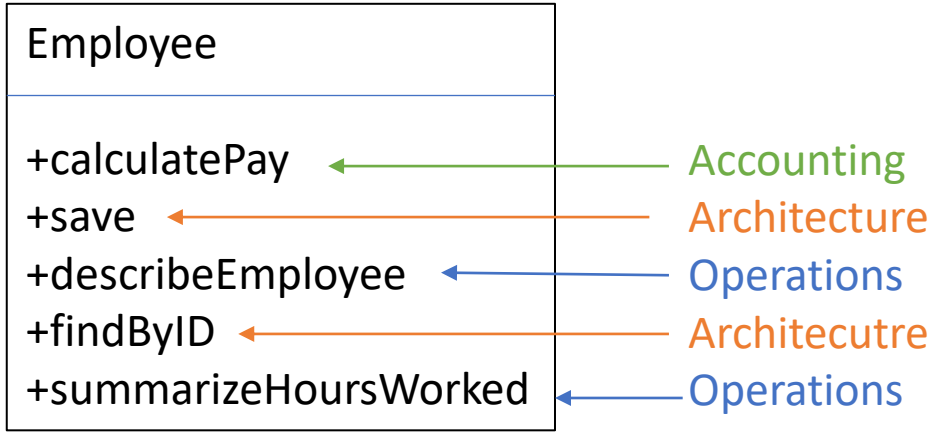
- A responsibility can be viewed as a source of change.
- The people served by that responsibility will request changes.

- Lawyers, managers, accountants
- Database architects (DBAs)
- Consumers of reports: clerks and accountants

It’s About Roles

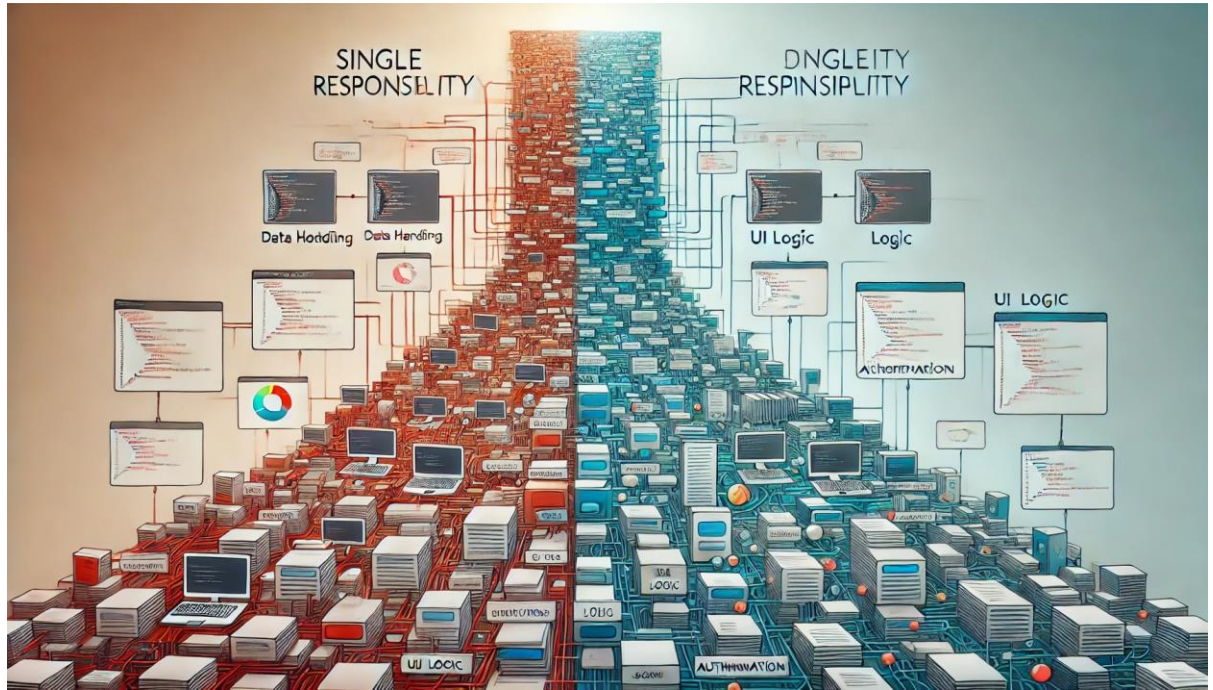
- Some people wear multiple hats.
- Best thing to do is separate the users from the roles (actors).

- For the Employee class there are three roles: accounting, architecture, and operations.



Responsibility Reprise

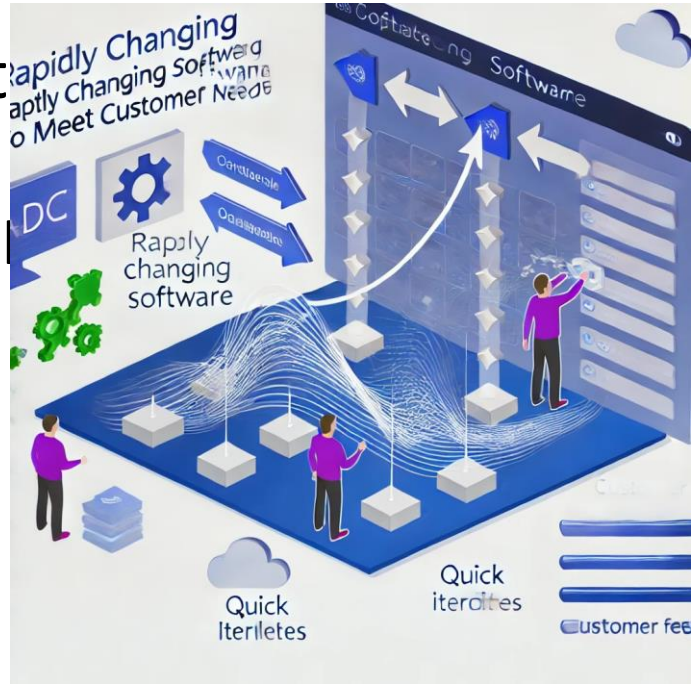
- A responsibility is a family of functions/methods that serves one actor.
- When the needs of that actor change, it serves as a source of changes for the family of functions/methods.



The Two Values of Software

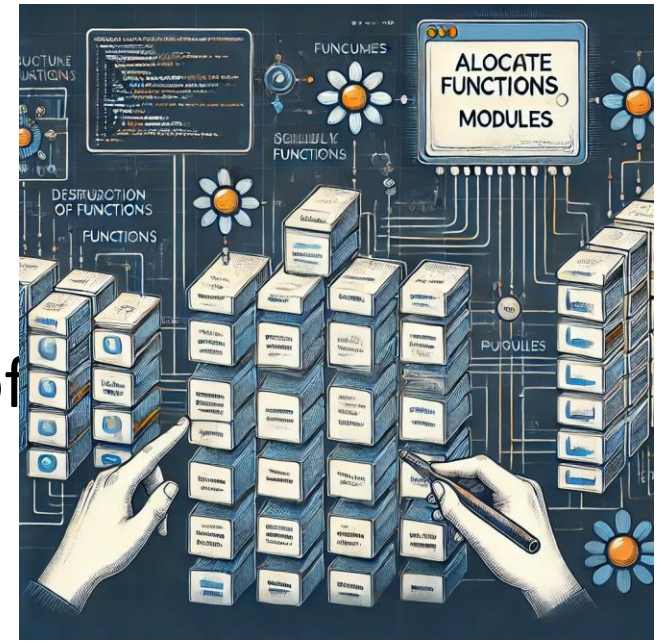
- Values can either save money or make money.
- Secondary Value: Behavior—no values, crashes, or de
  - The current software meets the needs of the current user
  - When the software needs change, the value of the softwa

- Primary Value: The ability to rapidly change the soft with the customer's changing needs.
  - If the software is difficult to change, then the primary val
  - Profitability is tied to the primary value of software.

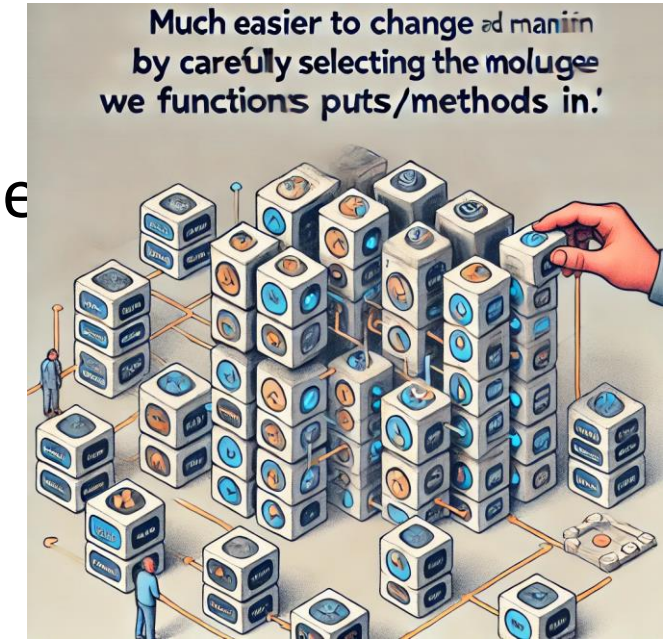


Careful Choices

- Allocate functions to modules – this is where much of value of software is achieved.

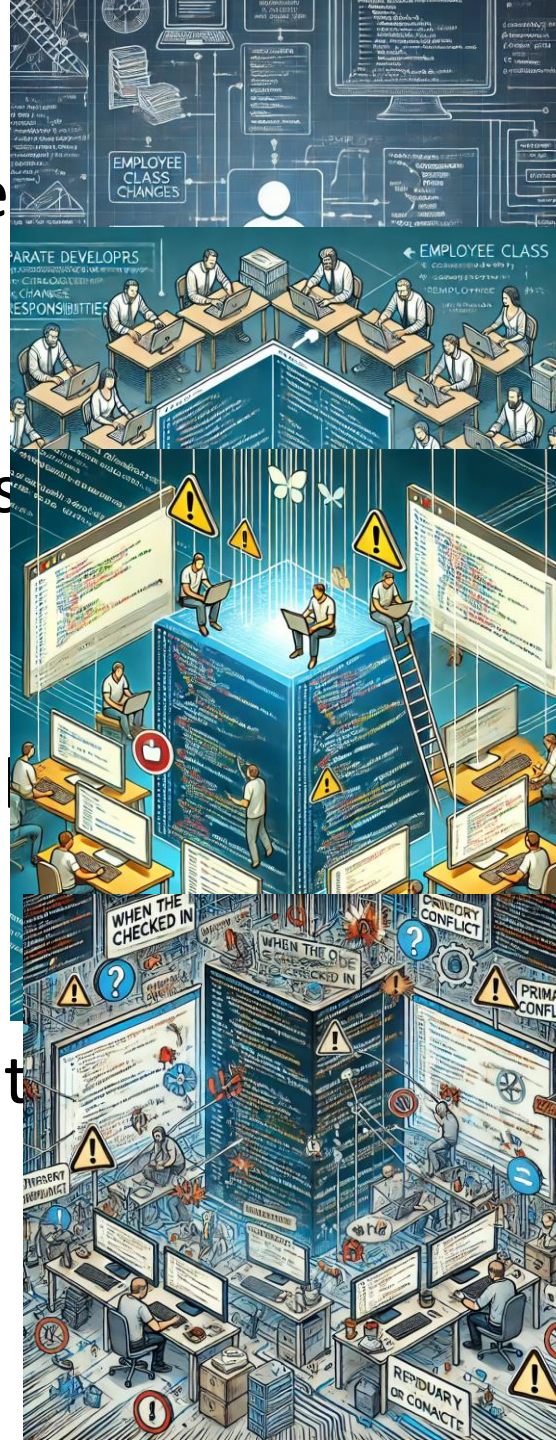


- Much easier to change and maintain by carefully sele modules we put of functions/methods in.

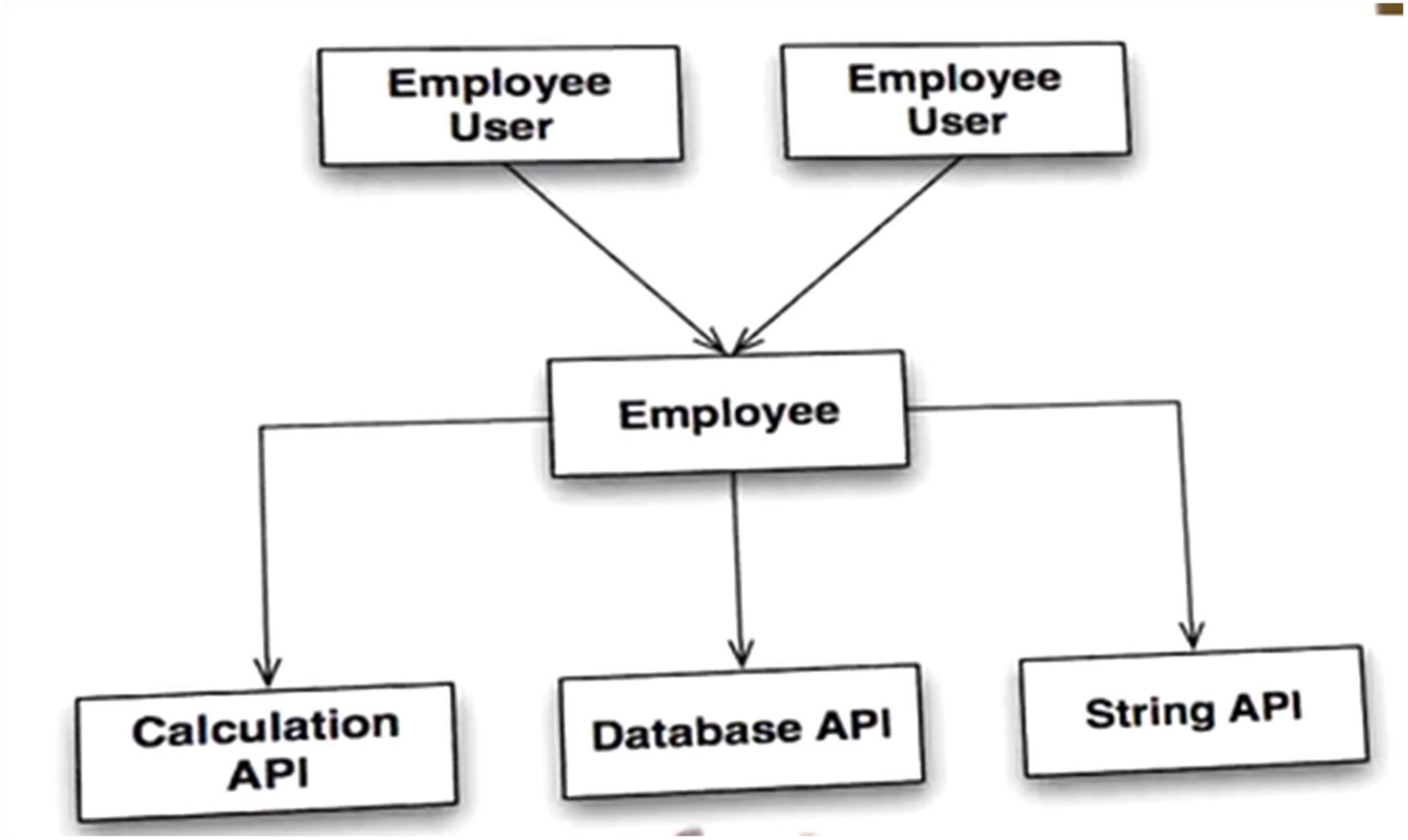


Conflict

- Suppose for the Employee class changes are needed in the architecture and policy.
- Now assume that separate developers are assigned to thos responsibilities.
- After the changes are made, when the source code is chec there is likely to be a collision, or conflict.
- This makes it more difficult to make changes, and reduces t value of the project.



Fan Out


Employee User
Employee User
Employee
Calculation API
Database API
String API

Adding a Report

- If you add a report to the Employee class, all users of the module need to recompile and redeploy.
- Business rules have been coupled to the reports.  **Bad!**
- Architecture has been coupled to business rules.  **Bad!**

Encroaching Fragility

- Other couplings tend to build over time.
- This causes **fragility** to appear.
- Customers and management then believe the developers are incompetent.
- Lions and Tigers and Bears, Oh My!

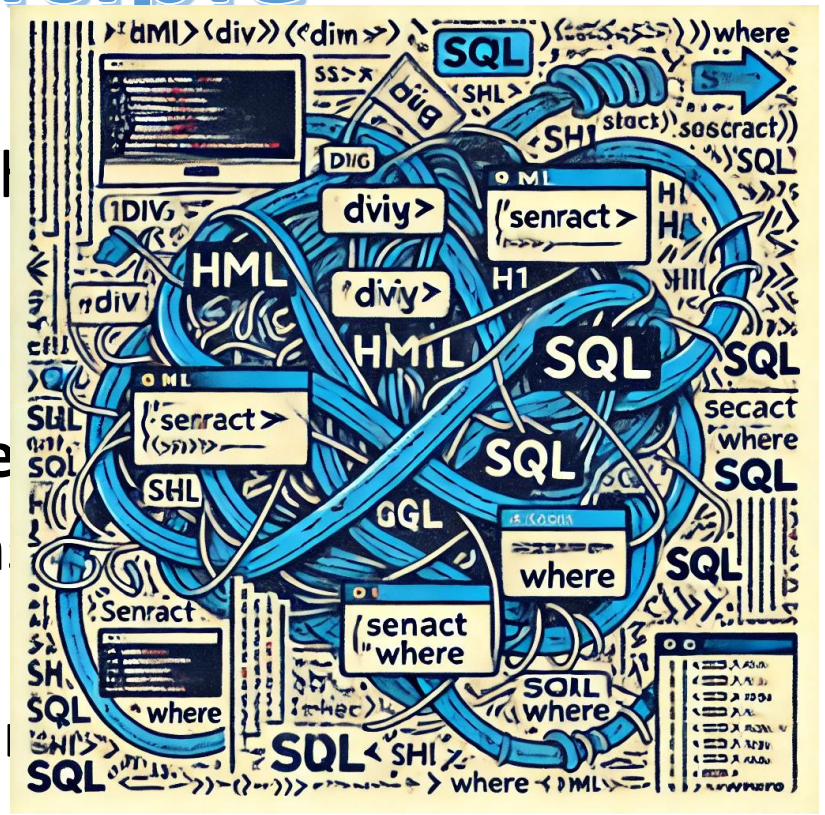
Single Responsibility Principle

- A module should have one and only **one** reason to cl
- **One** and only **one** responsibility.

- Gather together the things that change for the **same**
- Separate the things that change for **different** reason

- For instance, we do not mix **SQL** and **HTML** in the sa

- This principle is sometimes talked about in OOP class as **separation of concerns**.



SOLID

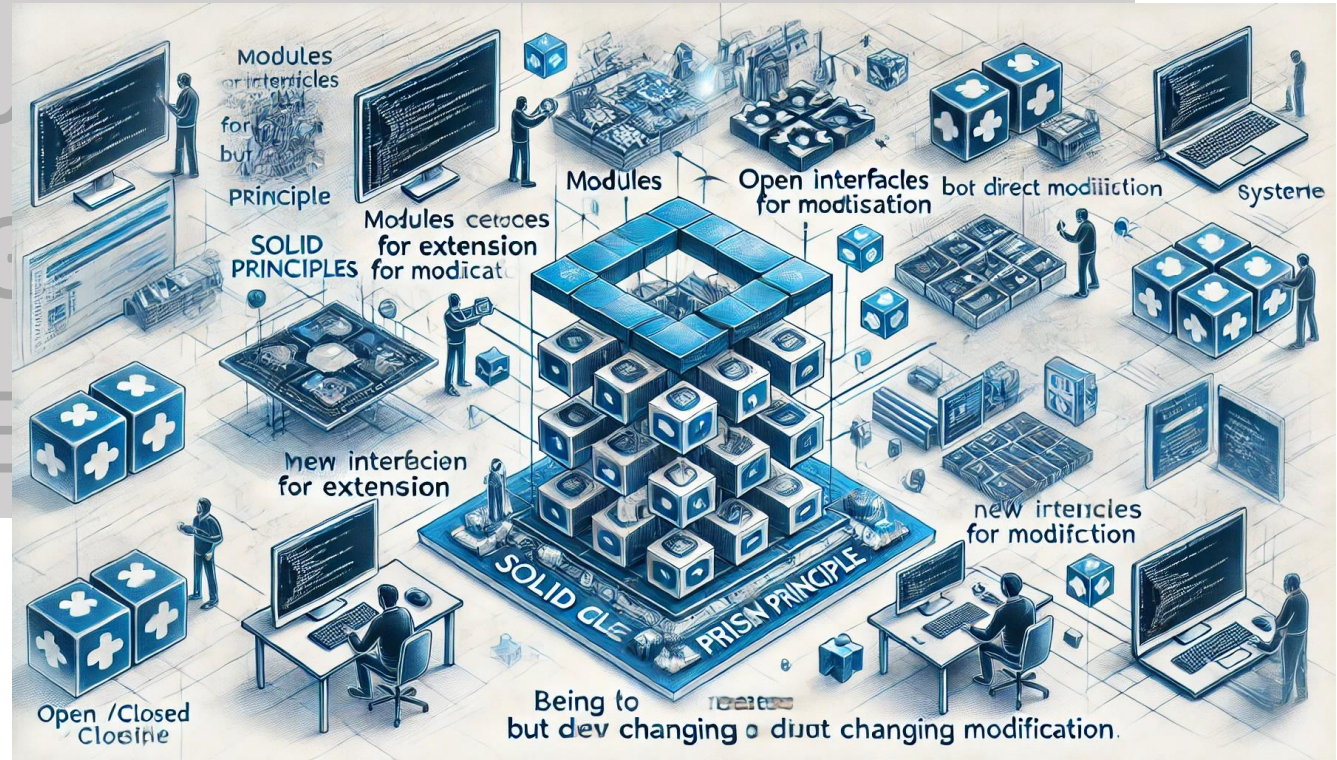
SINGLE RESPONSIBILITY PRINCIPLE

OPEN CLOSED PRINCIPLE

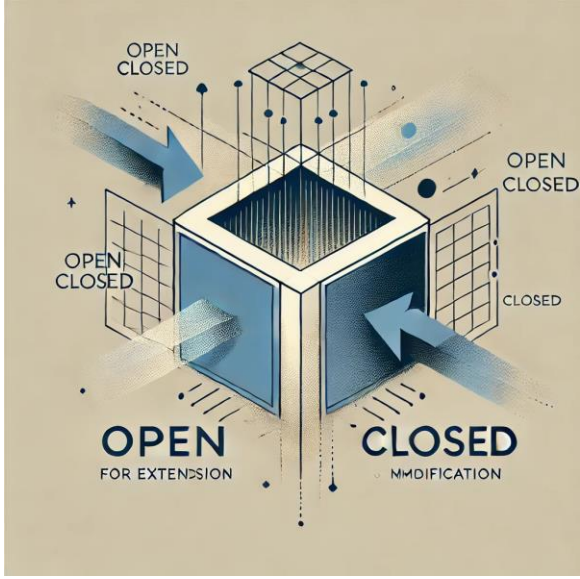
LISKOV'S SUBSTITU

INTERFACE SEGREG

DEPENDENCY INVE



Open Closed Principle



- A software module should be open for extension but closed for modification.
- Open means: very simple to change the behavior of that module.
- Closed means: the source code should not change.
- How?
- Remember the copy() method. You could change the behavior without changing the source code.
- We used abstraction and inversion (more about inversion later).

Point of Sale Example

```
void checkout(Receipt receipt)
{
  Money total = Money.zero;
  for item : items
  {
    total += item.getPrice();
    receipt.addItem(item);
  }
  Payment p = acceptCash( total );
  receipt.addPayment( p );
}
```

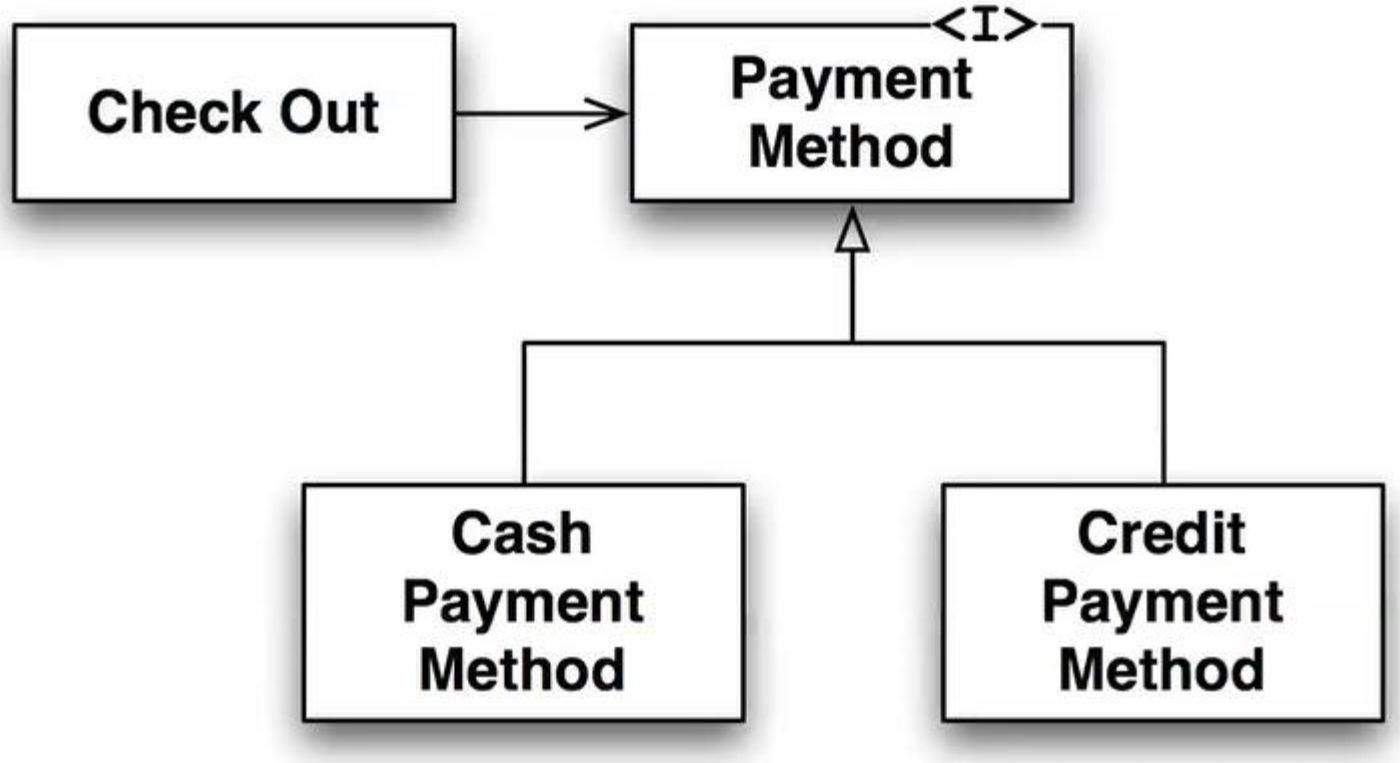
How About Credit Cards

```
Payment p
if( credit )
{
  p = acceptCredit( total );
}
else
{
  p = acceptCash( total );
}
receipt.addPayment( p );
```

- But now we modified the code.

Or Abstract

```
void checkout(Receipt receipt)
{
  Money total = Money.zero;
  for item : items
  {
    total += item.getPrice();
    receipt.addItem(item);
  }
  Payment p = pm.acceptPayment ( total );
  receipt.addPayment( p );
}
```


Check Out
<I>
Payment Method
Cash Payment Method
Credit Payment Method

Implications

- If you design a system that is open for extension b
  modification it means that every time you add a r
  by adding new code not by changing old code.

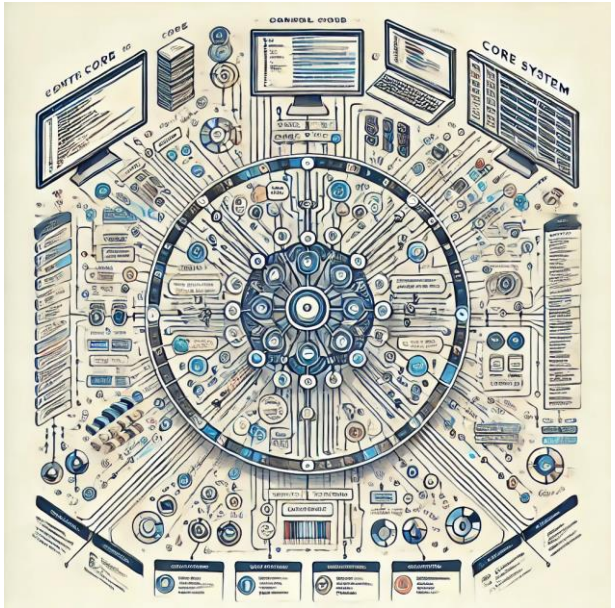
- If the old code never has to get modified then it c

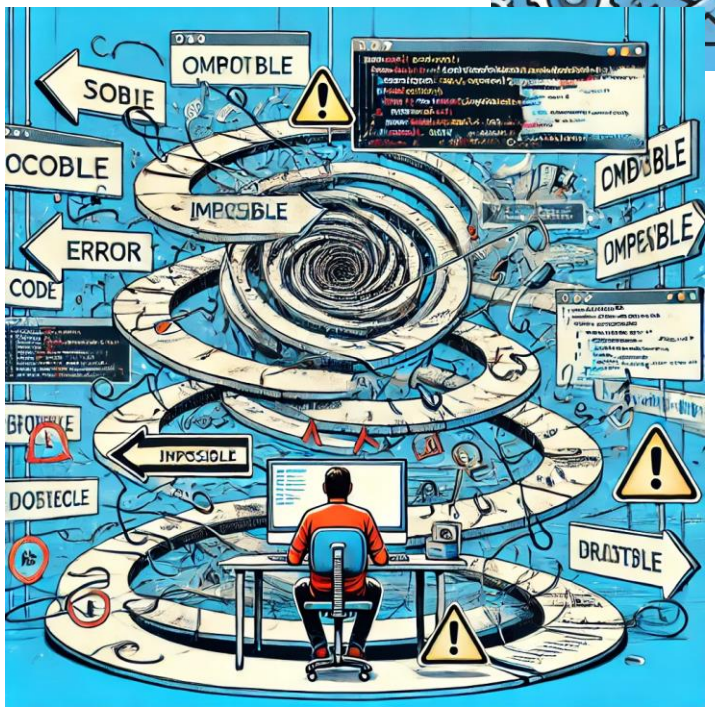
- You can write your modules cleanly and never have to worry about someone else messing them up.

- Code that conforms to Open Closed also conforms to the way customers think about software. When you add, you add and don’t need to alter existing code.

Is This Possible?

- Yes, theoretically.
- But it is hard, very hard.
- Sometimes not practical.
- How about main()?





Crystal Ball Problem

- We can’t really conform completely to Oper
- Should we just give up?
- No.
- It may be difficult to get systems to conform, but it’s much easier to get functions and classes to conform.
- The difficulty is related to the size of what you are doing.

SOLID

SINGLE RESPONSIBILITY PRINC

OPEN CLOSED PRINCIPLE

LISKOV’S SUBSTITUTION PRINCIPLE

INTERFACE SEGREGATION PRINCIPLE

DEPENDENCY INVERSION PRINCIPLE

LISKOV'S SUBSTITUTION PRINCIPLE

- Types support operations.
- We don't care what's inside of a type, but we care what it can do.
- I.E the functions and operations the types provide to us.

- A class is a bunch of methods and usually private data within.

Subtypes

- Is the following a type?

```
typedef point
{
  double x, y;
};
```

- No because it does not define (or imply) a set of operations.
- It’s just a data structure.

Transforming

- We can transform it into a type by providing a set of operations for it.

double distance(point *p1, point *p2);

double translate(point *p1, double dx, double dy);

double distance(point *p1, double angle);

- Now we have a type, especially if we hide the data and prevent anyone from using the data (except via the operations we provided).

Need getters and setters

```
void setx( point *p, double x);
void sety( point *p, double y);
double getx( point *p );
double gety( point *p);
```

Another Method

```
typedef describedPoint
{
  double x, y;
  char *description;
}
describedPoint *dp =  …;
Int x = getx((point *)dp);
```

Since first two members are the same as point, we can pass this into a function that expects a point type and x and y will be the same.

The relationship is asymmetric, and therefore illustrates a subtype. descriptedPoint is a subtype of point.

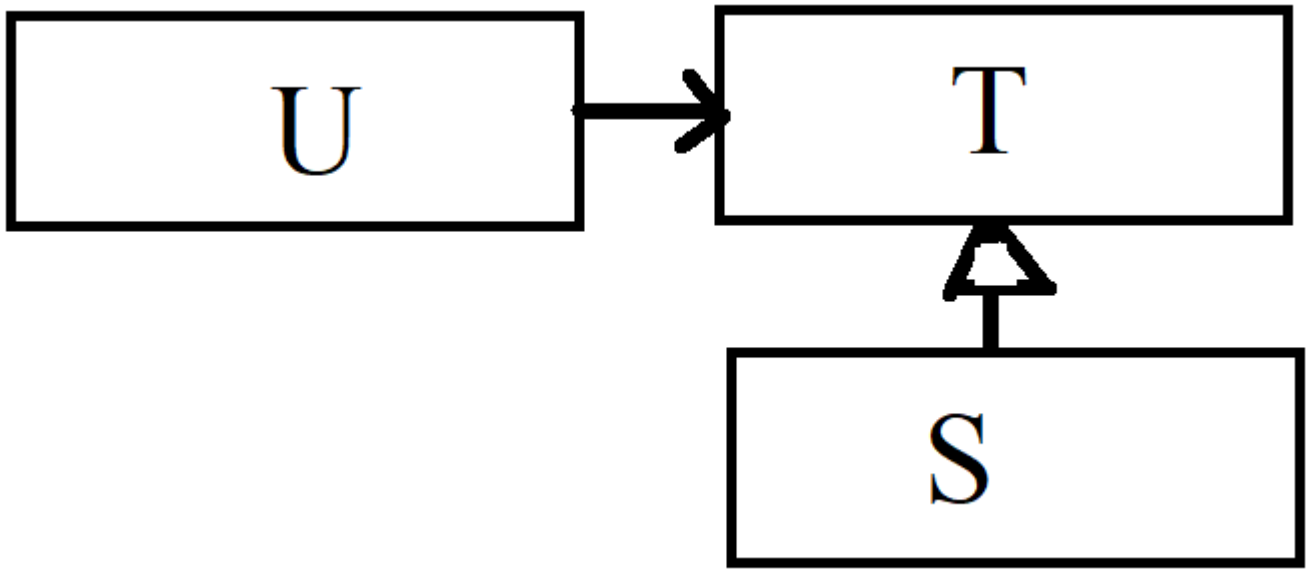
Think of floats and integers.

Leads to Classes and Inheritance

- C++, Java, C# inheritance can be traced back to this idea.

Liskov and Subtypes

- “What is wanted here is something like the following substitution property: if for each object o1 of type S there is an object o2 of type T such that for all programs P defined in terms of T, the behavior of P is unchanged when o1 is substituted for o2 then S is a subtype of T.” Barbara Liskov



More Explanation

- Subtypes can be used as their parent types.
- If you have a lot of users (U) and they can use a type (T)…
  - There is a subtype (S).
  - All of the users can use S without knowing it.

- It seems like a statement of what we know about polymorphism, but people still get this wrong at times.

Exchange Terminology

- Things are different between statically type polymorphism and dynamic. Remember the C++ virtual methods.

- Instead of invoking methods we can say that we **send messages**.
- This is the same meaning but a different connotation.

- At runtime, the ability to send a message (call a method) is determined.
- If OK, it is completed -- -otherwise a runtime error results.

Remember What a Type Is

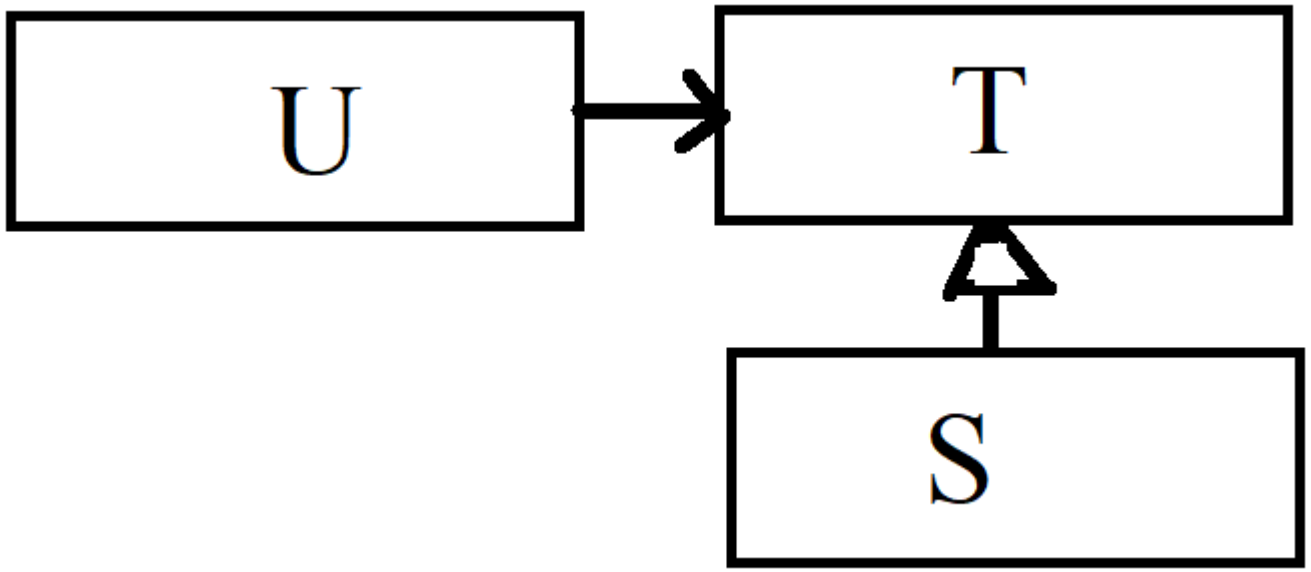
- A type is a bag of methods.
- Since a subtype is a type, it is also a bag of methods.
- To de substitutable with its parent, it must have the same methods as its parent – even if they have different implementations.

- In statically-typed languages we use inheritance to achieve this.
- In dynamic languages we achieve it by writing the same methods into the subtype.

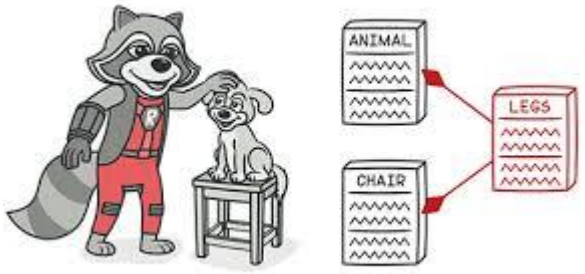
- In either case the subtype is substitutable for its parent.

Inheritance

- For the remainder of this discussion, we will substitute inheritance with subtype.



Refused Bequest



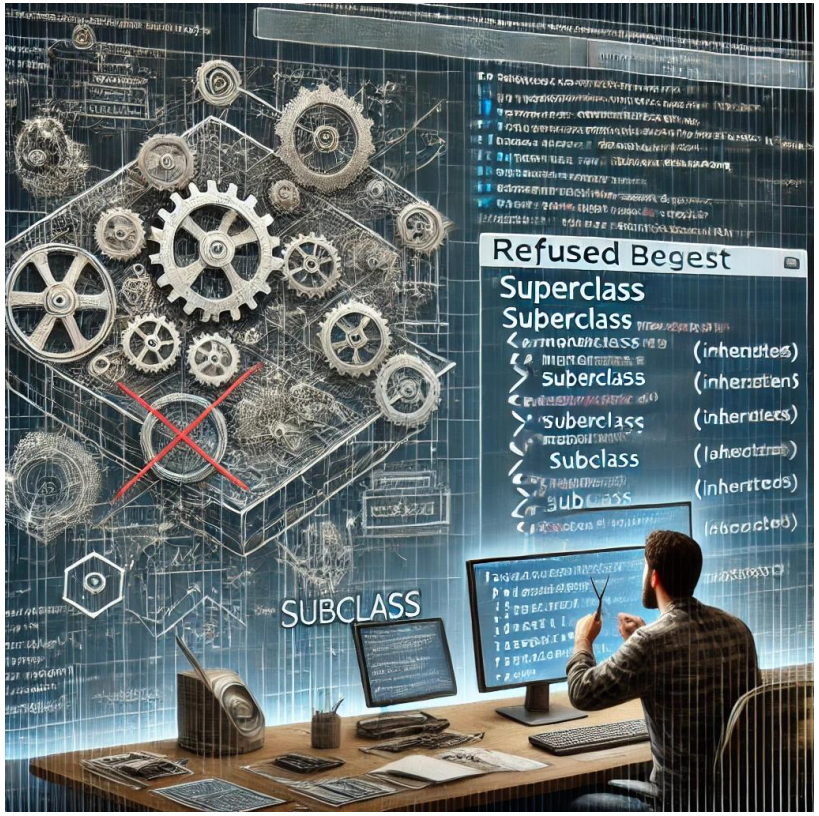
- When the Liskov substitution principle is violated, the result is usually a **refused bequest**.

- This occurs in dynamic programs when you send a message to an object and the object does not have the requested method. This usually **throws an exception**.

- This can also occur when a method of the subtype does something that the client of the supertype does not expect.
- For example: the subtype might throw an exception that the base class does not expect.
- Another example: the subtype might cause a side effect that the base class does not expect.

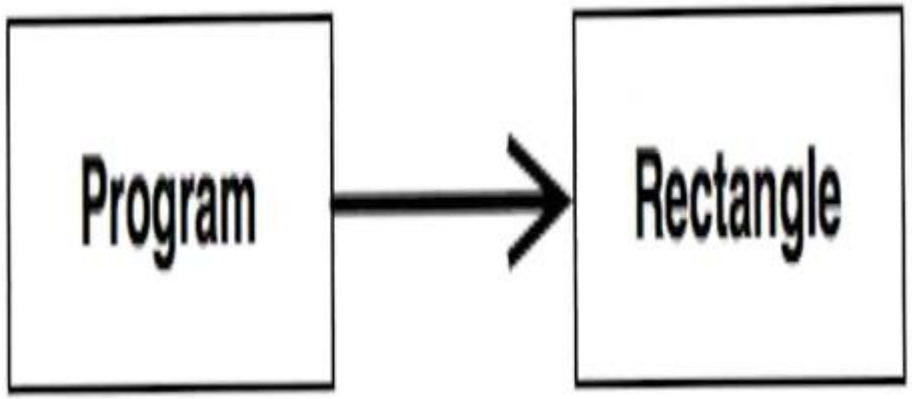
Can Occur in Static and Dynamic

- A refused bequest can happen in both static and dynamic objects.

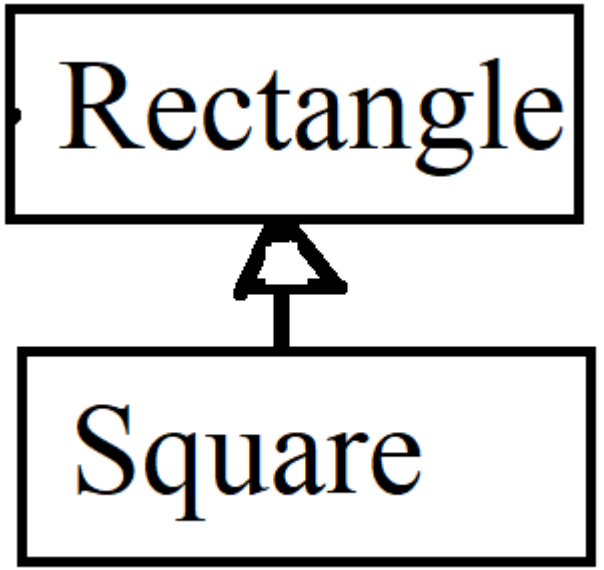


Classic Case

```
public class Rectangle {
    private double height;
    private double width;
    private Point topLeft;

    public double area();
    public double perimeter();

    public double getHeight();
    public double getWidth();

    public void setHeight(double height);
    public void setWidth(double width);
}
```



New Requirement: Square



- Square is a Rectangle
- But Rectangle has width and height while

  Square only has side.
- Square also inherits setHeight() and setWidth()
- Square also inherits getHeight() and getWidth()
- We can override to preserve “Squareness”.

```
public class Square : Rectangle
{
  public:
    void setWidth( double value )
    {
        base.setWidth( value );
        base.setHeight( value );
    }
};
```

- Now a rectangle behaves like a rectangle and square a square.

What If?

- Somewhere in our program a call to setWidth() is made to a class that it thinks is a Rectangle but is really a Square?
- But if a Square gets passed into the module, then when the module changes the Side width and height change.

- These mixups can produce “undefined behavior”.
- It might wait to crash when you ship to a customer.

Latent OCP Violation

- You could use an instanceof operation to determine if an object is a Rectangle or a Square. This is a dependency.
- But this violates the Open/Closed principle.
- Every refused bequest, every violation of Liskov Substitution, is a latent violation of the Open/Closed principle.
- In order to repair the damage caused by the refused bequest, we need to add if statements and introduce dependencies.
- Once violating Open/Closed, the software gets rigid and fragile.

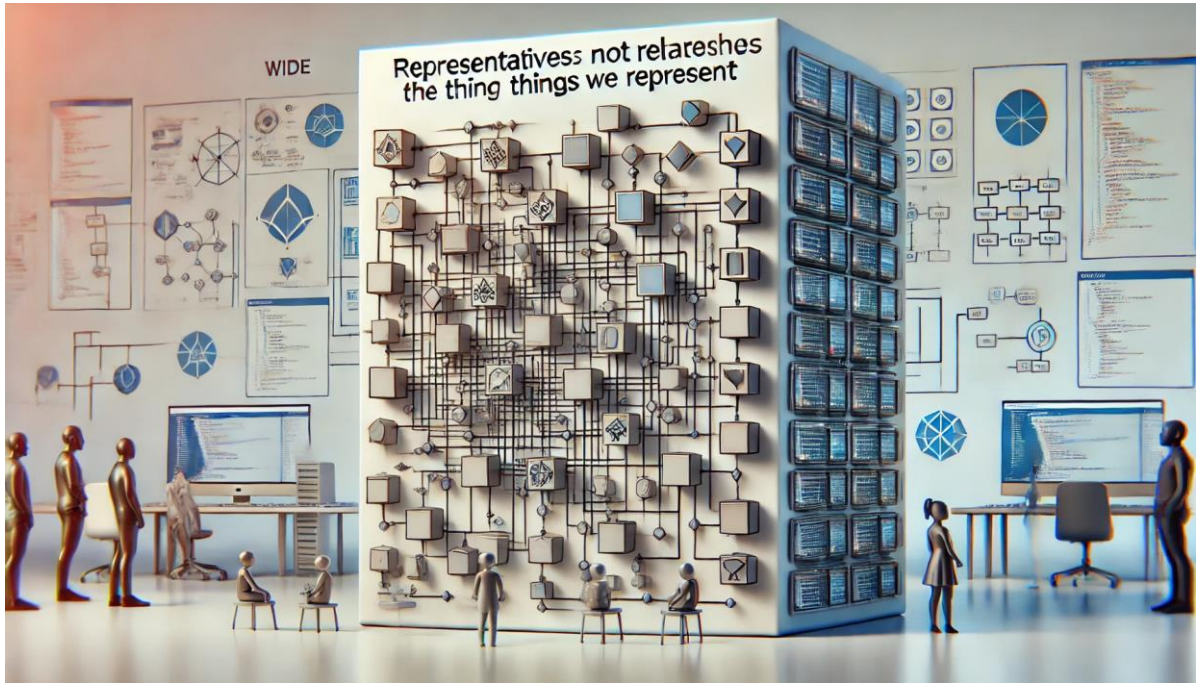
- This reflects poorly on the developers.

Solution

- How can we represent both Rectangle and Square?
- Maybe make Square the base class?
- Maybe make the objects immutable?
- **You will need to treat Square and Rectangle as different types.**

Representative Rule

- Representatives do not share the relationships of the things they represent.
- While geometrically a Square is a Rectangle, the software representatives don’t share the subtype relationship.



Numbers

- Consider **integer**
- Also consider **real**
- Integers are subtypes of real.

- Also, every real number is a **complex** number.
- Real is a subtype of complex.

- But a complex contains two real numbers.
- This example shows that some examples make sense in the real world but not in code.

Lists

```
class T { … }
class S extends T { … }

List<T> ts = new List<S>();
// Nope, incompatible types.
```

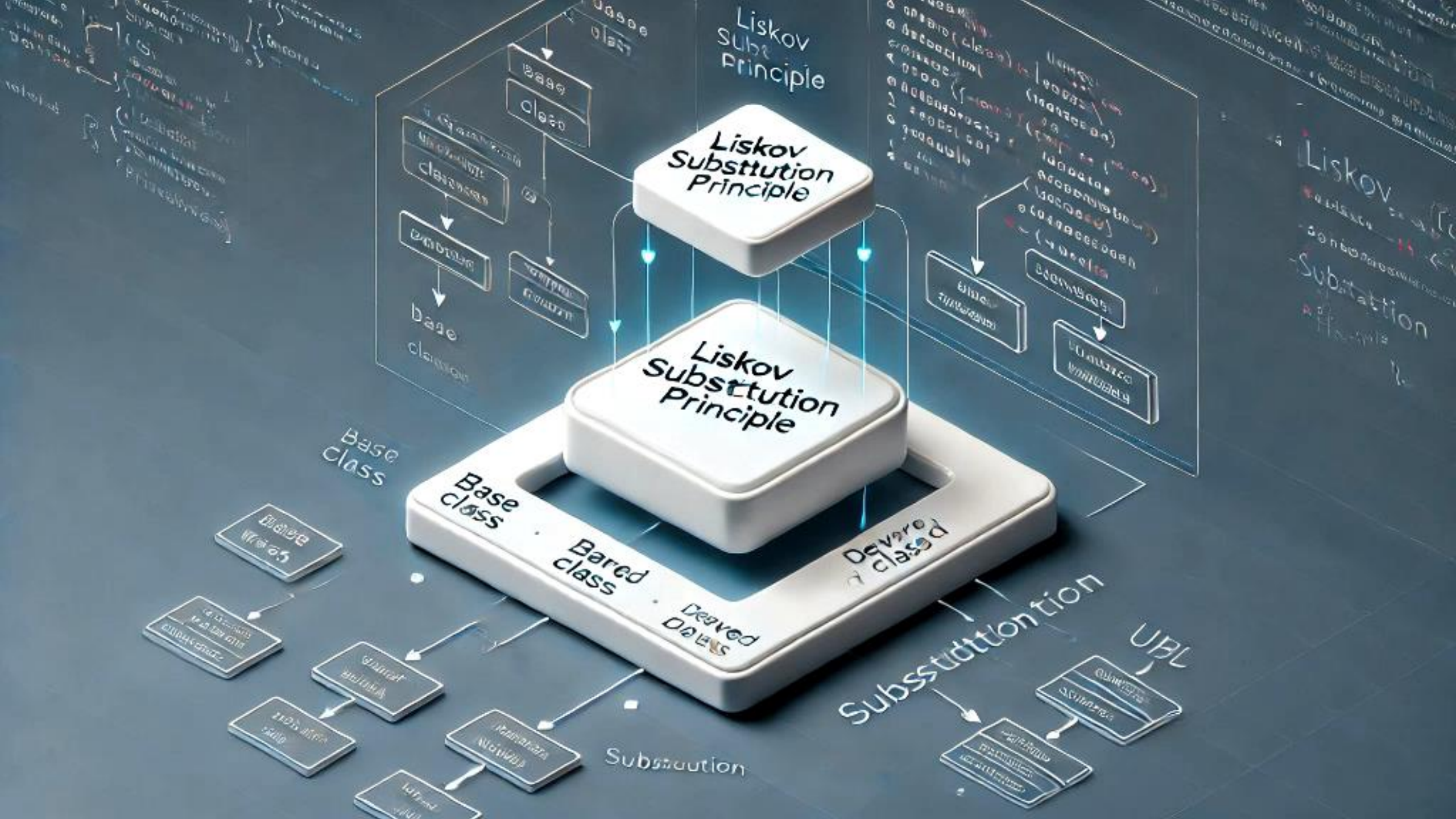
Example

Suppose Circle is a subclass of Shape.

```
void g()
{
  List<Circle> circles = new ArrayListCircles();
  f(circles);
}

void f(List<Shape> l)
{
  l.add( new Square() ); // This might put a square in this list.
}
```

Heuristics

- If the base class does something, the derived class must do it too. (And in a way that it does not violate the expectations of the callers.)
  - You can't take expected behaviors away from a subtype. A subtype can do more than its parent, but it can never do less.
  - If you have a degenerate function that does nothing such as void f(){} then you might have a Liskov Substitution Principle violation, especially of those functions were implemented in the base class. This is not a hard/fast rule, just be suspicious.
  - A more blatant clue is when a derived function is written to unconditionally throw an exception. The author of the derived function does not want you to call it. This might call for an **if instanceof** statement.


Liskov
Substitution
Principle
Base
class
Derived
class

SOLID

SINGLE RESPONSIBILITY PRINCIPLE

OPEN CLOSED PRINCIPLE

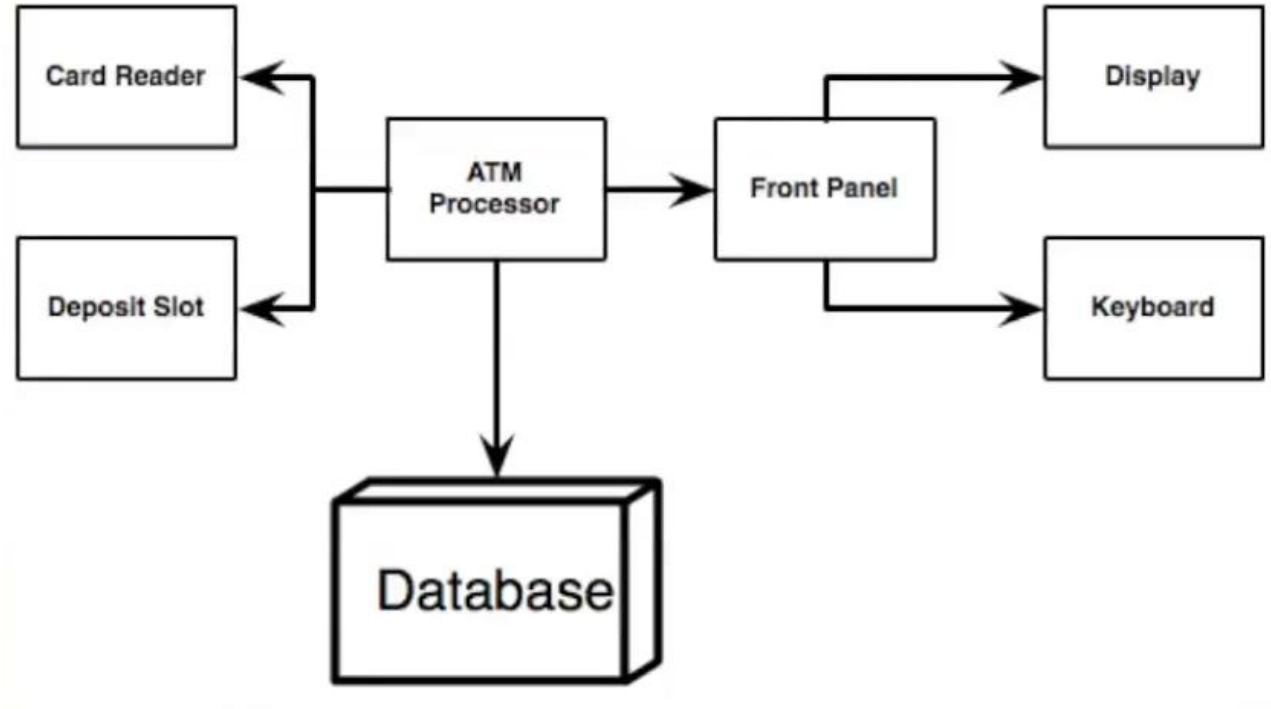
LISKOV'S SUBSTITUTION PRINCIPLE

INTERFACE SEGREGATION PRINCIPLE

DEPENDENCY INVERSION PRINCIPLE

INTERFACE SEGREGATION PRINCIPLE

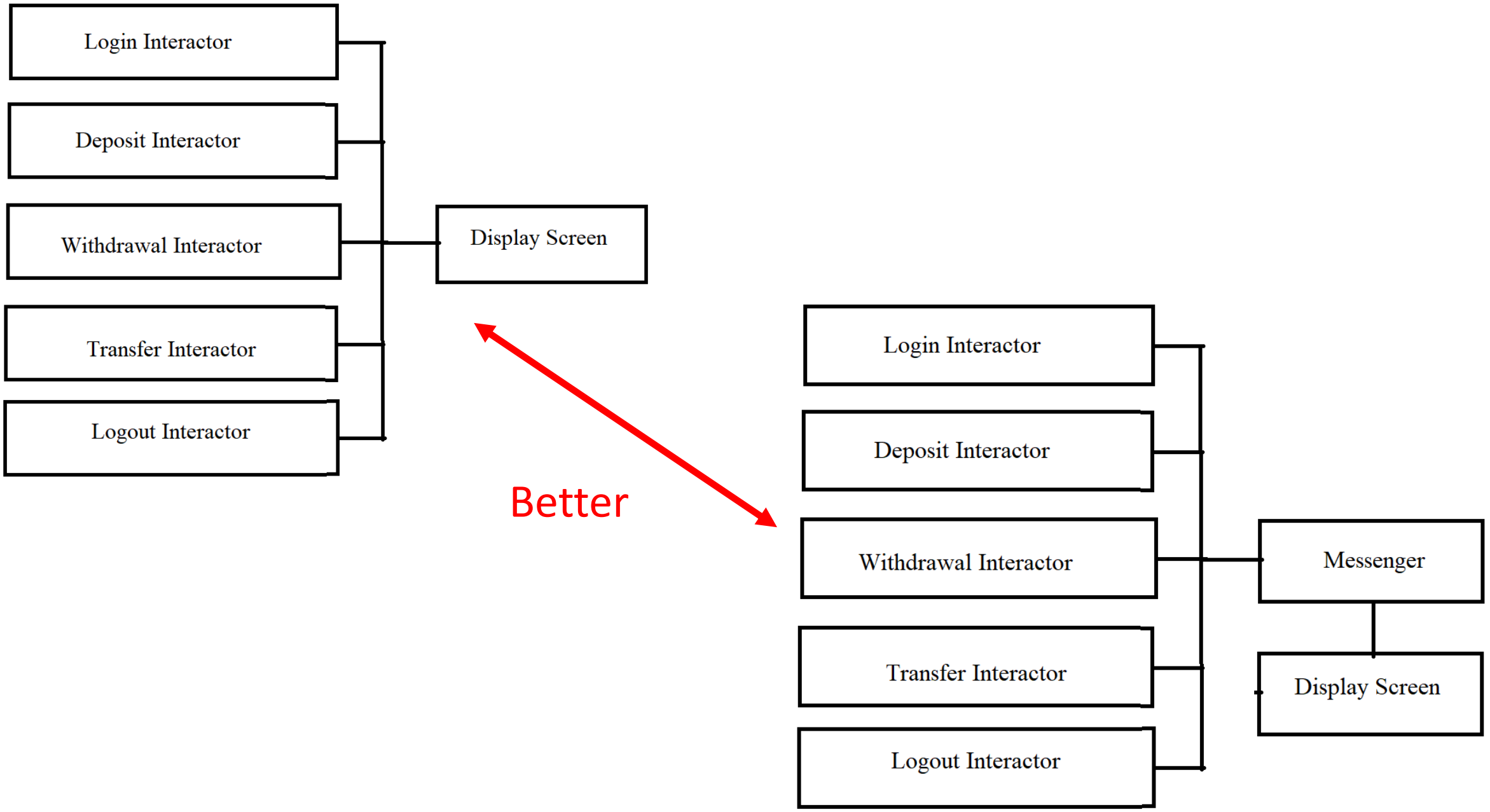
- Design for ATM as an Example. Consider the following:



Use Cases

- Login
- Logout
- Withdraw
- Deposit
- Transfer

- Now must create Interactors for each use case.


Login Interactor
Deposit Interactor
Withdrawal Interactor
Transfer Interactor
Logout Interactor
Display Screen
Better
Login Interactor
Deposit Interactor
Withdrawal Interactor
Transfer Interactor
Logout Interactor
Messenger
Display Screen

Messenger Methods

tellUser

If a string needs to be formatted for display, this violates the single responsibility principle.

More on this later, but first some background.

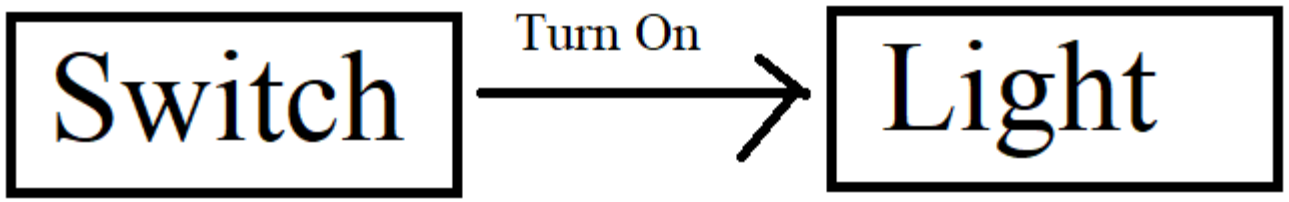
Getting Started with ISP

- Problem: given a light switch and a light, design the software that will turn on the light when the switch is activated.

Objects in the Design

- Should electricity be an object in the design?
- Should the bulb or base or electric cord be objects in the design?

- The simplest solution might be to have the switch object send a **Turn On** message to the light object.



What Might Be Wrong With That?

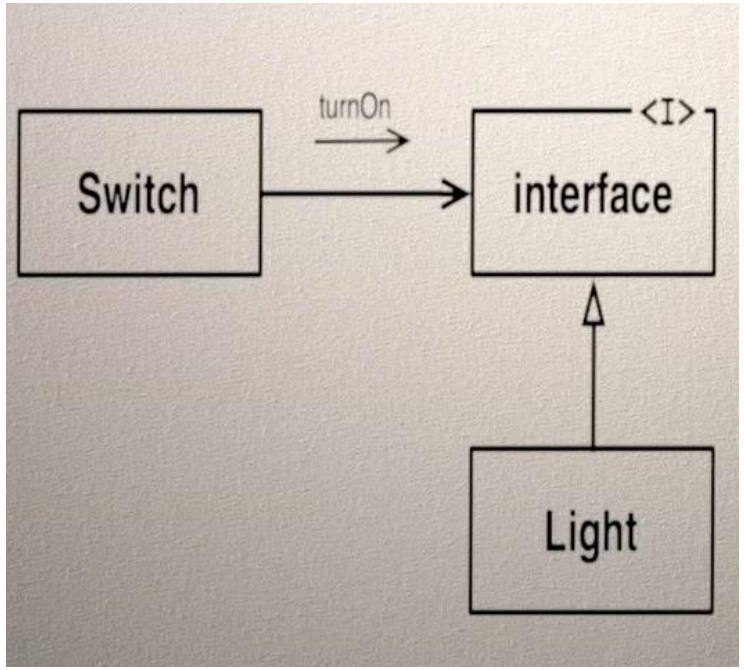
```
public class Switch
{
  private Light light
  public Switch( Light light)
  {
    this.light = light;
  }
  public void activate()
  {
    light.turnOn();
  }
}
```

Notice that the Switch class contains the name of the Light class.

Why should the switch know anything At all about the light?

This Can Be Fixed

- Insert an Interface between the Switch and the Light.
- The Interface will have the turnON() method, the Light will implement the interface, and the Light will supply the implementation for the turnOn() method.



Name?

- But what should be name the interface?
- How about AbstractLight?
- Should Light and AbstractLight be shipped in the same component?
- This is naïve.
- Inheritance is a strong relationship, so the assumption is that these things with a strong relationship should remain together.

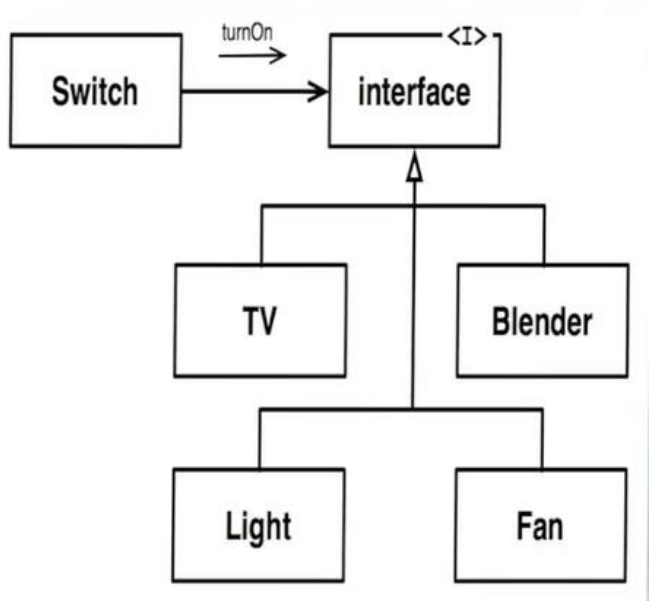
- But this introduces a dilemma for statically-typed object-oriented languages.
- Inheritance, while it is the strongest of the physical coupling, is the weakest of the logical couplings.

It’s The Switch

- It’s the Switch that has the tight logical coupling to the interface.
- The Switch can’t do its job without the interface.
- On the other hand, the Light considers the interface to be a nuisance. It doesn’t care about the interface it just drags it around.
- The interface doesn’t even know the Light exists.
- The Switch and the interface should be packaged together.
- The Light and the interface have a loose logical coupling but a tight physical coupling.

Best Deployment

- Deploy the Switch and interface in one component and the Light in another.
- They can be deployed separately.
- The Light component is now a plugin for the Switch component.



What Should the Name of the Interface Be?

- TurnOnAble
- ActivateAble
- **SwitchAble**

- Represents the tight logical binding of the interface to Switch.
- Interfaces have more to do with the classes that use them the with the classes that implement them.

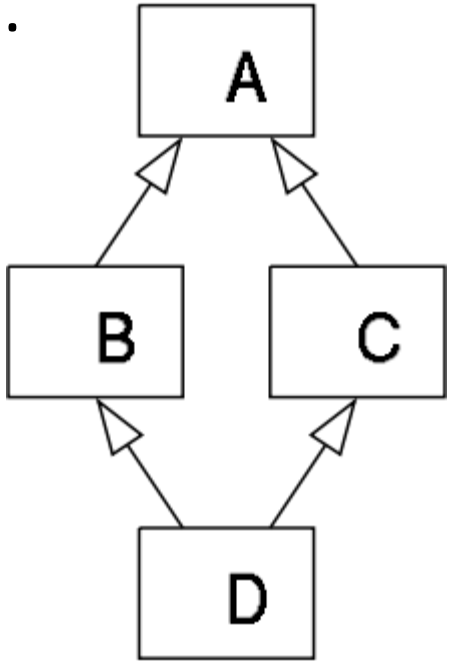
What is an Interface

- An abstract class with nothing but abstract methods.
- In C++ you don’t need an interface because you can create classes with all abstract methods.
- But in C# and Java you need a specific interface.

- The reason for the need for interfaces is that the language inventors did not know how to implement fully abstract classes—they punted.

Multiple Inheritance

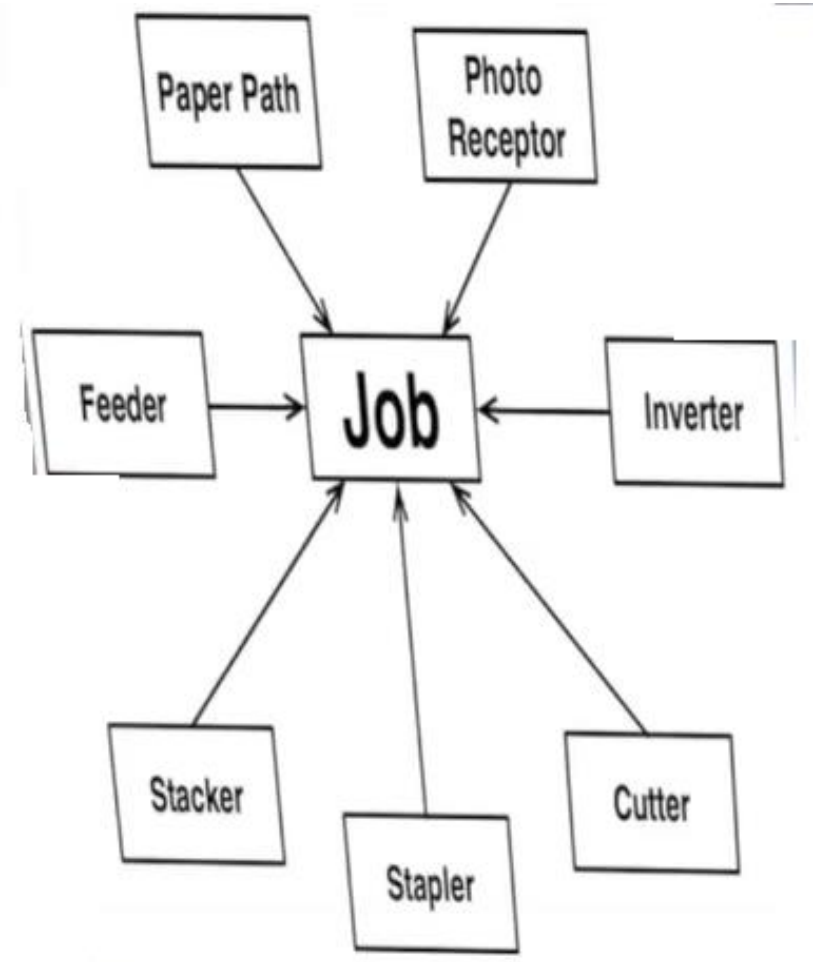
- MI can be useful in many situations.
- In C# you can multiply inherit interfaces but not classes.
- This is so that the implementers could avoid the “deadly diamond of death”.



The "diamond problem" (sometimes referred to as the "Deadly Diamond of Death") is an ambiguity that arises when two classes B and C inherit from A, and class D inherits from both B and C.

Fat Classes

- Take, for instance, a high-end copier.
- The controlling program must know when to engage the rollers to feed paper, snap the light, add staples, and many other things.
- Let's now say that the main class is called Job.
- It fans out significantly.



Enormous Amount of Information

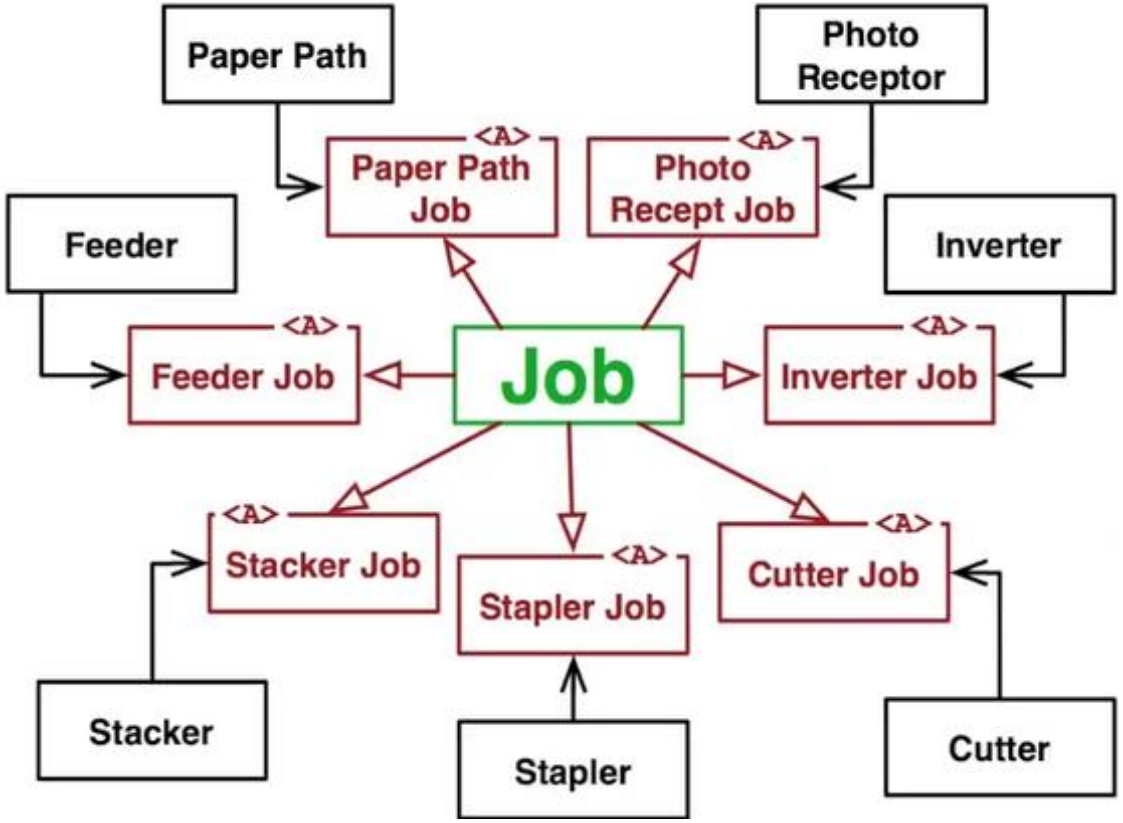
- Each subsystem needs an enormous amount of information to do its job.
- For this reason, the Job class has probably hundreds or thousands of methods.
- Because of the large number of dependencies this program would take a long time to build.

Solving The Problem

- The subsystems that depend on the Job class depend on it for different reasons.
- The number of methods they call and the methods themselves are different.
- There is a very small overlap.

Insert

- Insert abstract classes between the base classes and the Job class.
- Now the Job class can multiply inherit from these abstract classes.
- Now, when adding new methods to the Job class the only subsystems that must be rebuilt are those that call them.
- Now builds are much faster



Fat Classes Cause Another Problem

- The coupling can be so tight and significant that it makes it difficult to separate code into multiple modules.
- Start by isolating the Fat Class from its clients by creating interfaces.
- Make sure the interfaces contain only the methods that the clients will call.
- Then, multiply inherit them into the original Fat Class.

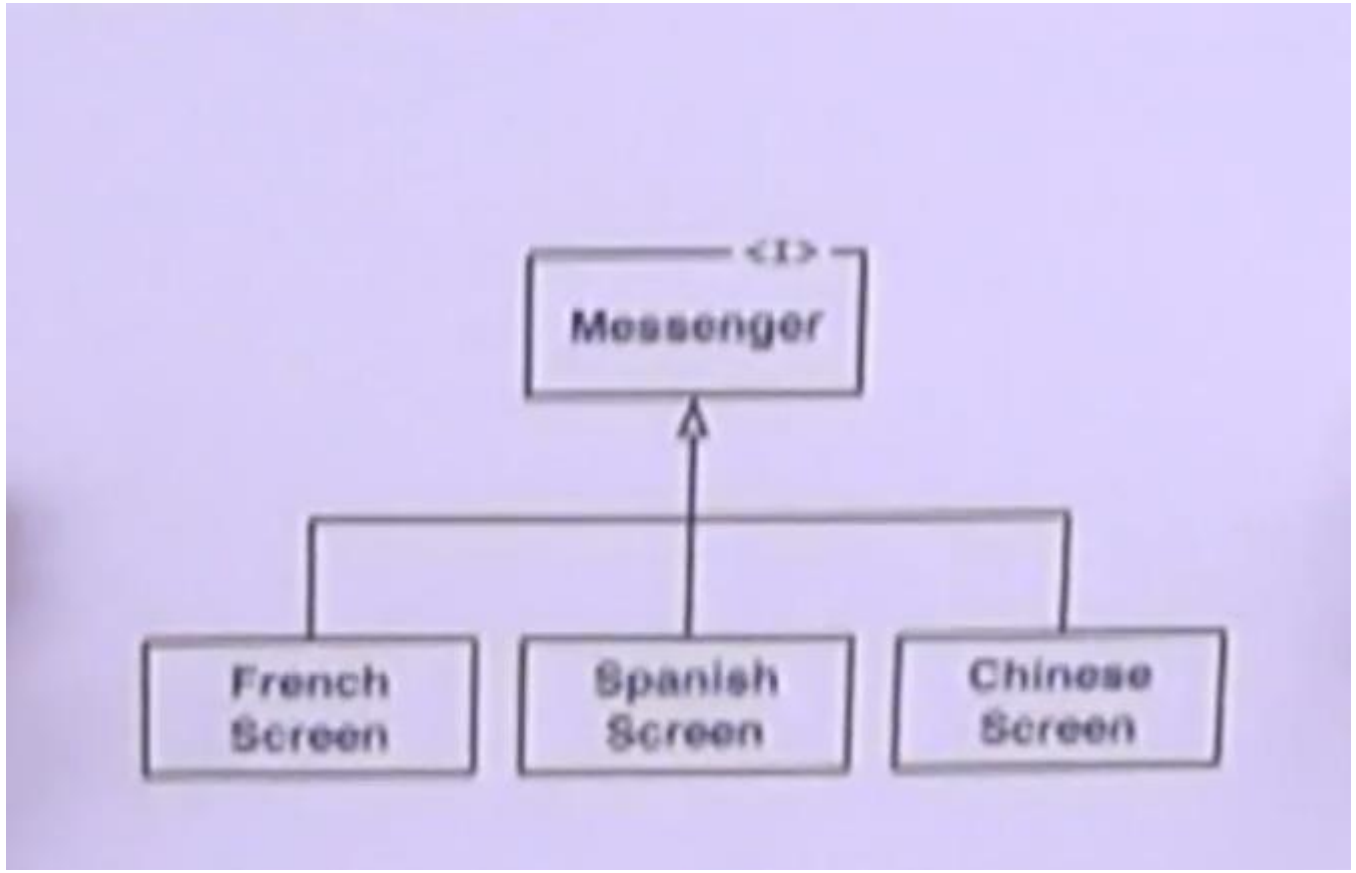
ATM Example

- What methods for the messenger interface?

- askForCard
- tellInvalidCard
- askForPin
- tellInvalidPin
- tellCardWasSeized
- askForFUnction
- askForAccount
- tellNotEnoughMoneyInAccounty
- tellAmounttDeposited
- tellBalance

What About Different Languages

- Different languages into different subclasses.



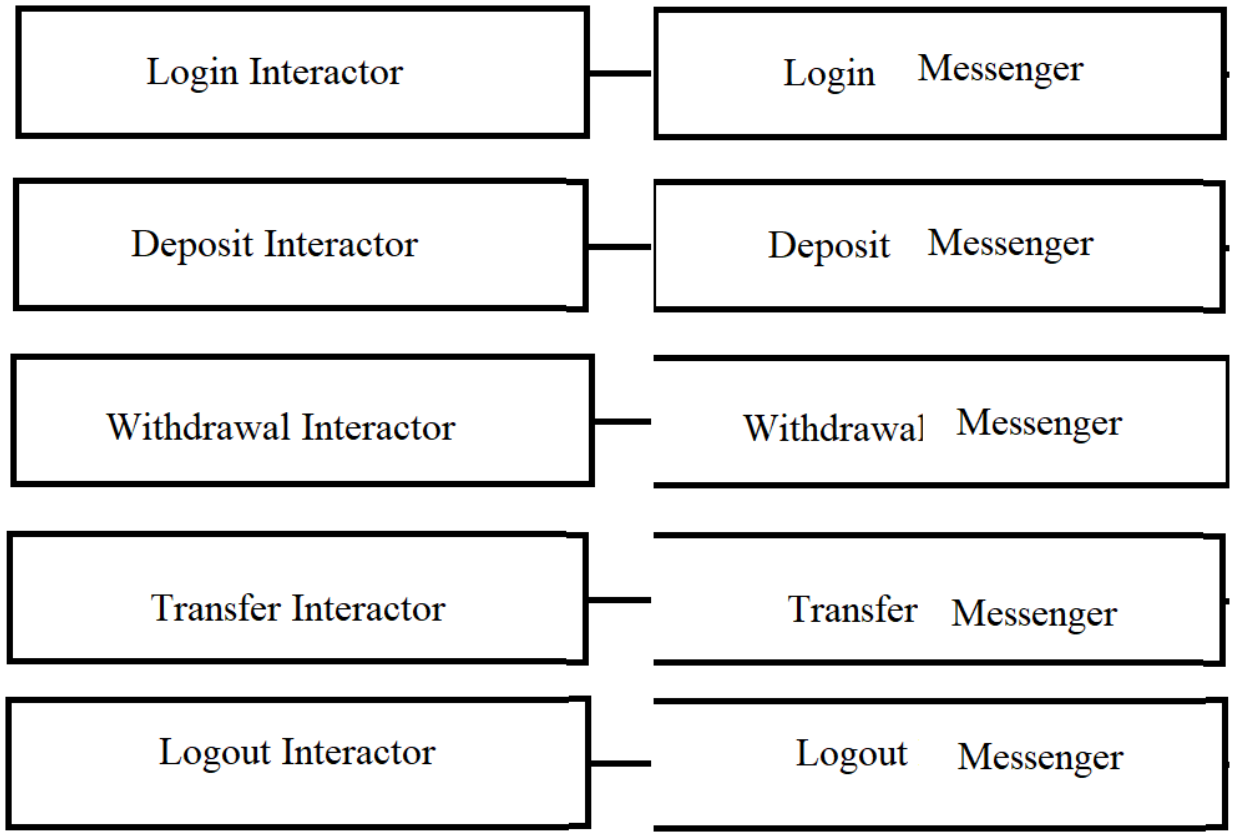
Principles Used

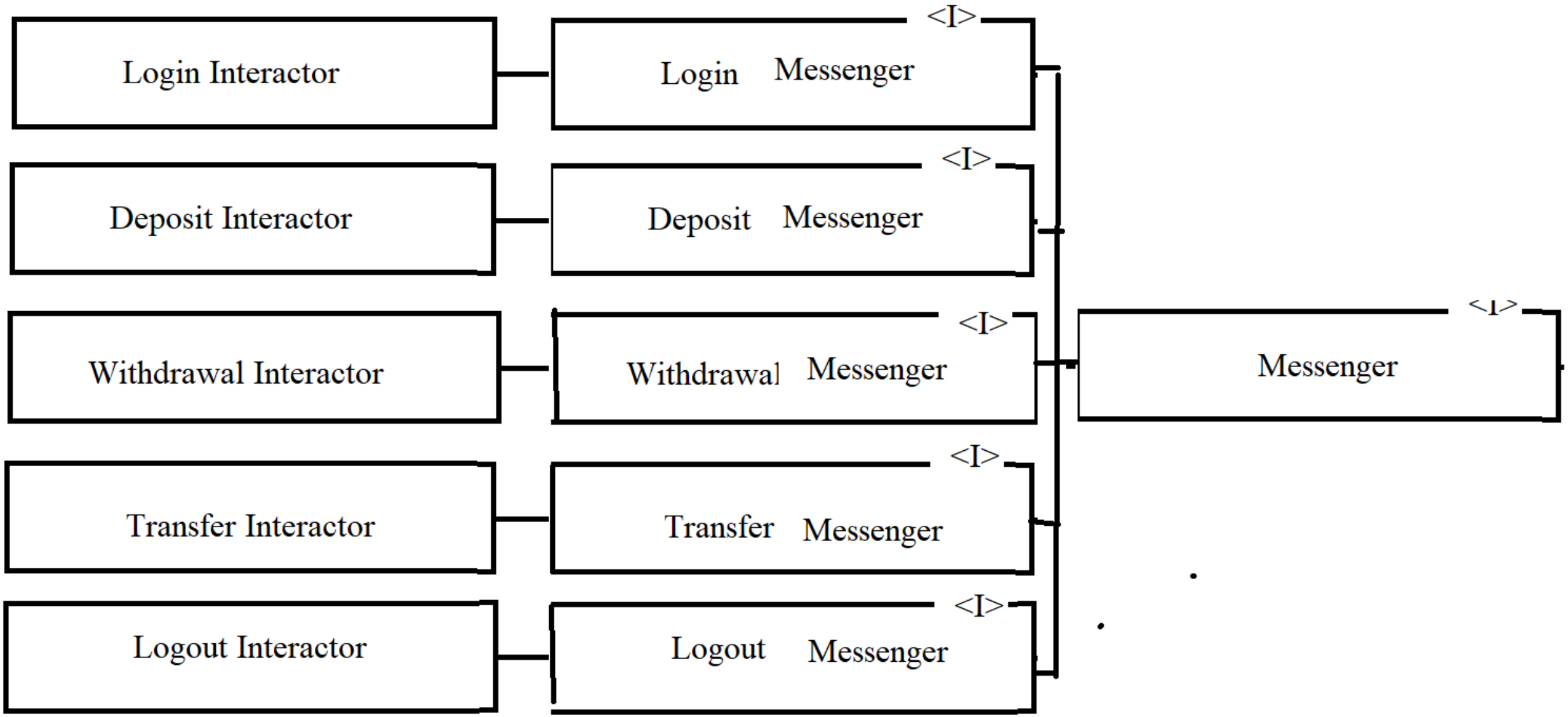
- Open / Closed Principle
- Liskov Substritution Principle

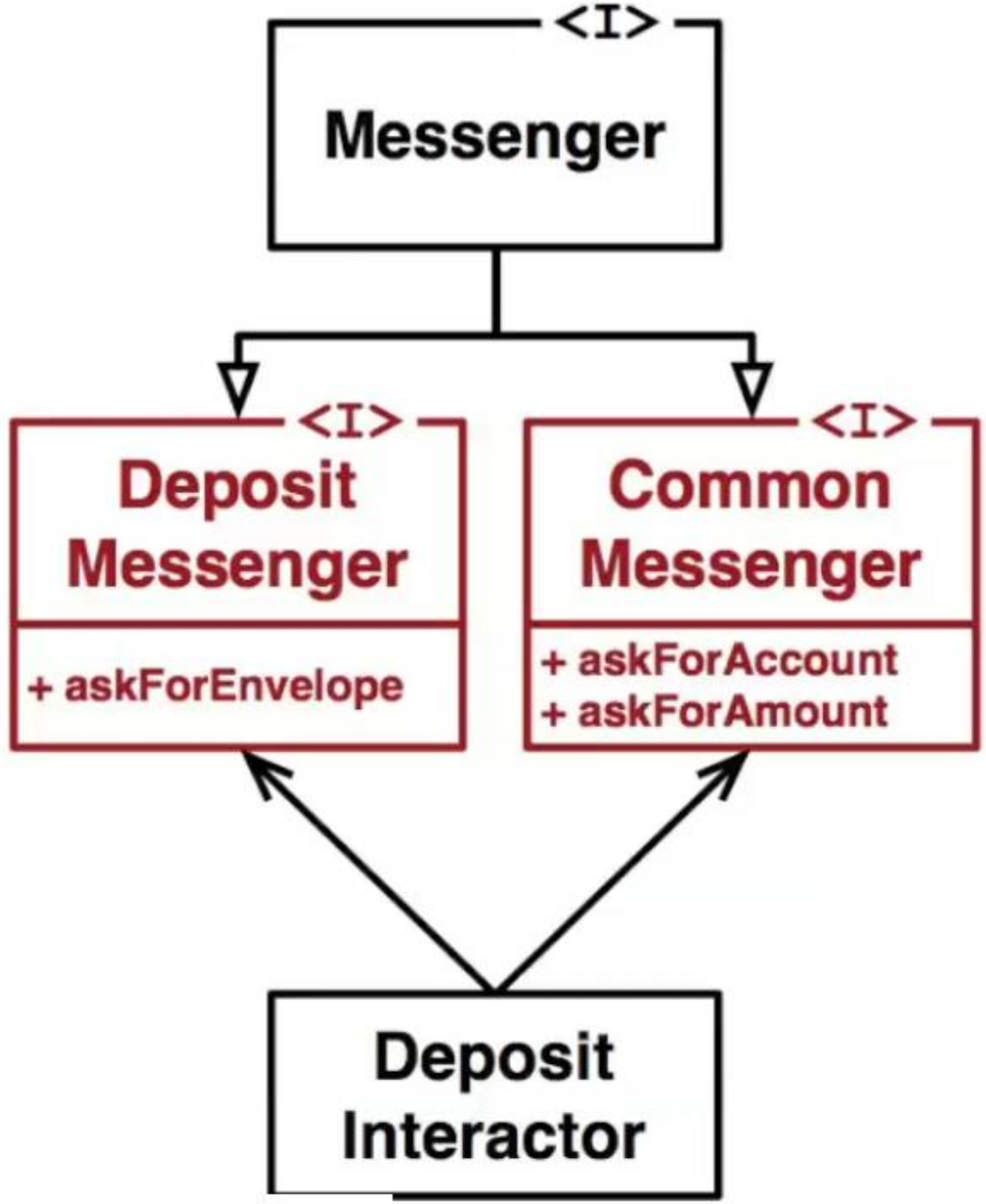
Would This Be Hard to Maintain

- What is customer wants to add another step to the withdrawal use case.
- After they withdraw, you need to display a message that they will be charged a user fee. Need user approval.
- If you add a method to the messenger class, the Open / Closed principle is violated. Now all interactors must be recompiled.
- This makes the system rigid and fragile. Many interactors implement methods they never use.
- Remember that the ask for Pin and Card fall into the same category.
- This backwards dependency ties all interactors together.

Move Methods into Separate Classes




Login Interactor
Login Messenger
<I>
Deposit Interactor
Deposit Messenger
<I>
Withdrawal Interactor
Withdrawal Messenger
<I>
Transfer Interactor
Transfer Messenger
<I>
Logout Interactor
Logout Messenger
<I>
Messenger
<I>


<I>
Messenger
<I>
Deposit
Messenger
+ askForEnvelope
<I>
Common
Messenger
+ askForAccount
+ askForAmount
Deposit
Interactor

SOLID

SINGLE RESPONSIBILITY PRINCIPLE

OPEN CLOSED PRINCIPLE

LISKOV'S SUBSTITUTION PRINCIPLE

INTERFACE SEGREGATION PRINCIPLE

DEPENDENCY INVERSION PRINCIPLE

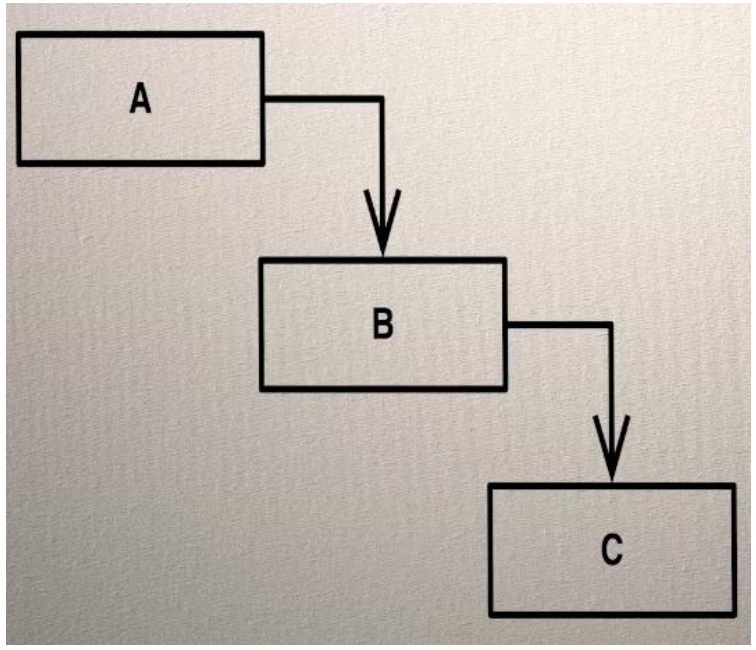
DEPENDENCY INVERSION PRINCIPLE

- We will consider two types of dependencies to consider:
  - Runtime: two modules interacting to call functions and/or access variables.
  - Compile Time: Function and variable names are dependent at compile time.

- Source code dependencies come at a significant cost.
- When code is compiled and refers to a name in another module, it must learn he details of what that name implies.
- For instance: class A depends on class B, when class A is compiled it must find the details of class B.

Compilation Dependencies

- This means you can’t simply compile one module in isolation.
- You must compile all modules that a module depends on.
- And module B might rely on module C.



Fast Compilers Hide This

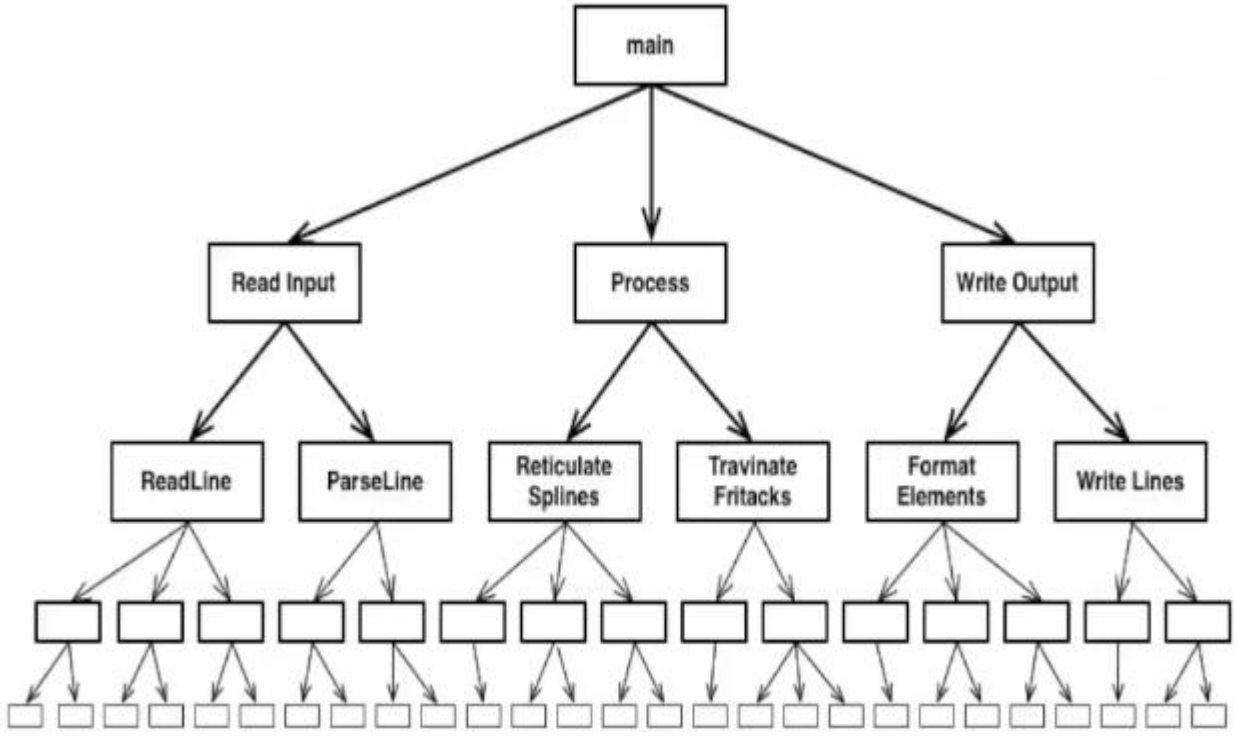
- Compilers today are fast and the dependency compiles cannot always be seen.

- But deployment can be another issues that does make a difference.
- A lot of modules might need to be deployed.

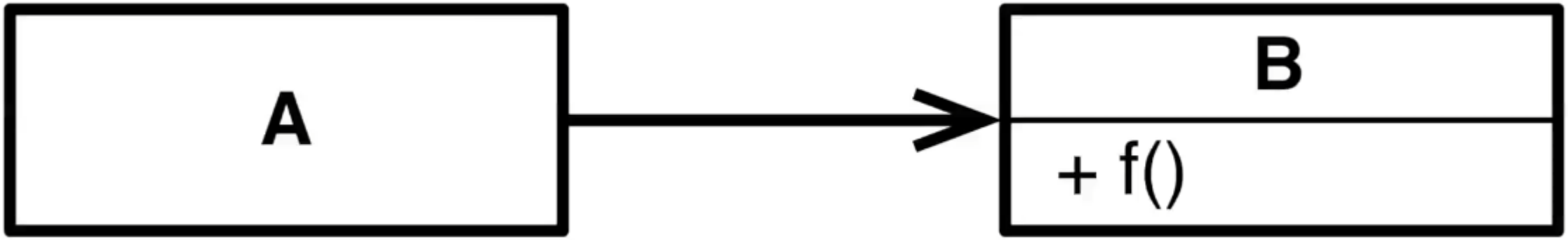
- Also, what if there are many teams.
- One team might make a change that forces other teams with their dependencies to recompile.
- This requires minimizing impacts between teams.

Structured Design

- This is a top-down methodology: start with main
- Design subroutines that main should call.
- Then design the subroutines that those subroutines should call.

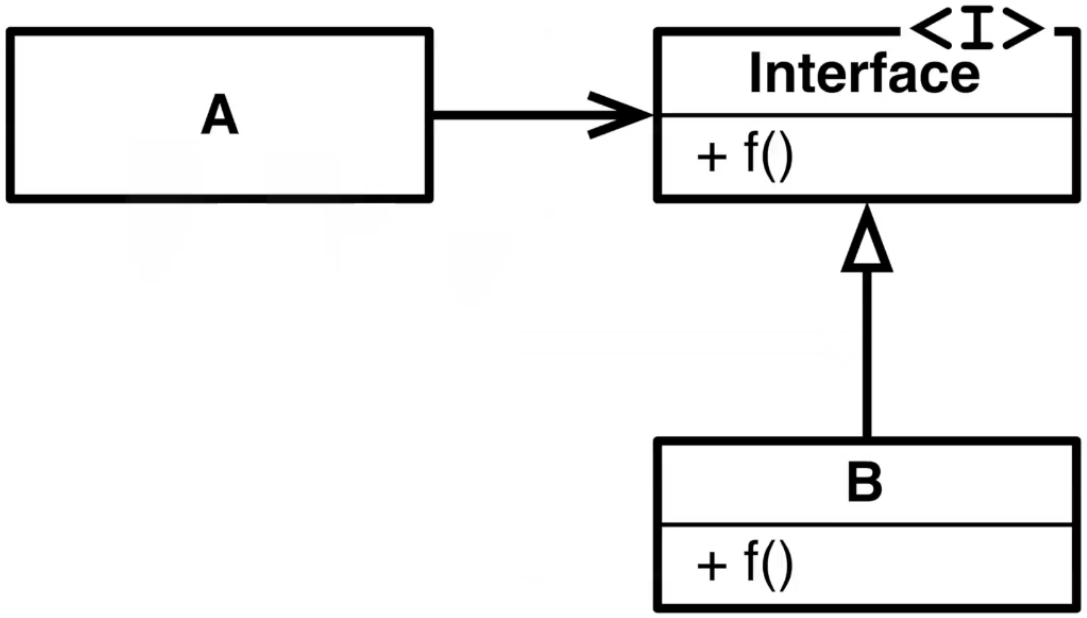


Keeping Dependencies From Crossing Boundaries



- This has a compile time and a runtime dependency.
- To deal with this dependency we can use polymorphism.

Polymorphic Solution

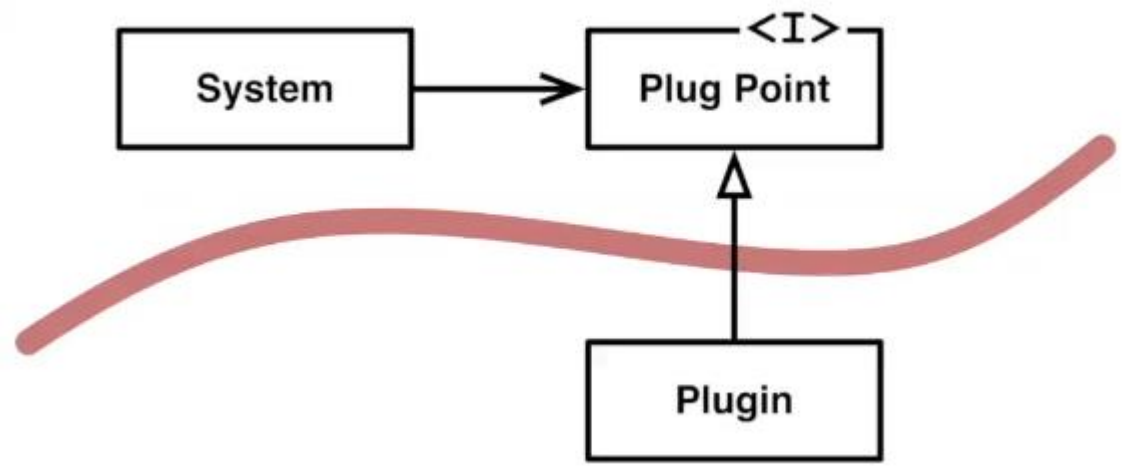


- This is now a runtime dependency but not a compile-time dependency.
- Thus, dependency inversion is introduced.

When?

- Dependencies are inverted whenever the source code dependency oppose the direction of the flow of control.
- This creates boundaries between source code modules.

- Boundaries are used to create plugins. (The caller has no idea of who or what it is calling.)



Architectural Implications

- High level policy should not depend on low level detail.
- Low level detail should depend on high level policy.

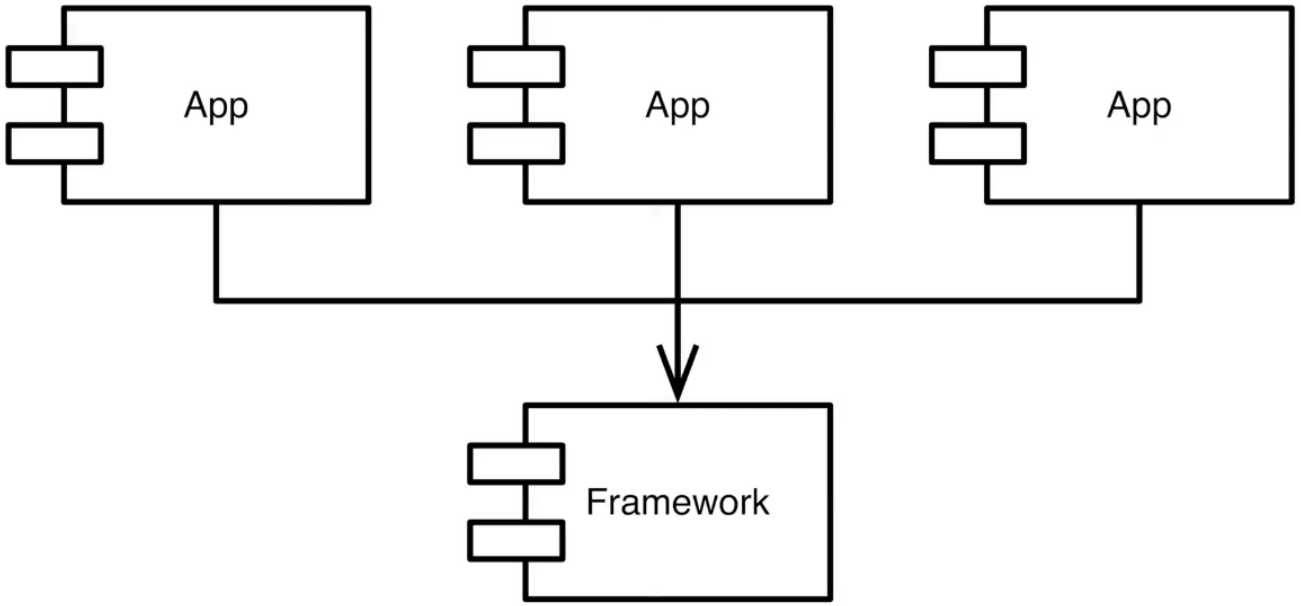
- Modules that contain high level policies (such as use cases) should not depend on low level modules such as those containing (accessing) databases or web formatting.
- There is an apparent dilemma since the high level modules eventually need to access a database.
- The solution is to invert the dependency.
- For instance, the database layer can become a plugin to the use case.

A Reusable Framework

- This might include a system that needs to employ large amounts of reuse.
- A reusable framework can provide multiple modules/programs a set of plugins that each can use.

- This can be difficult.
- You should always build the reusable framework in parallel with at least two or three higher level programs that will use it.

The Inversion



- The apps should all depend on the framework, but the framework should not depend in any way on the apps.

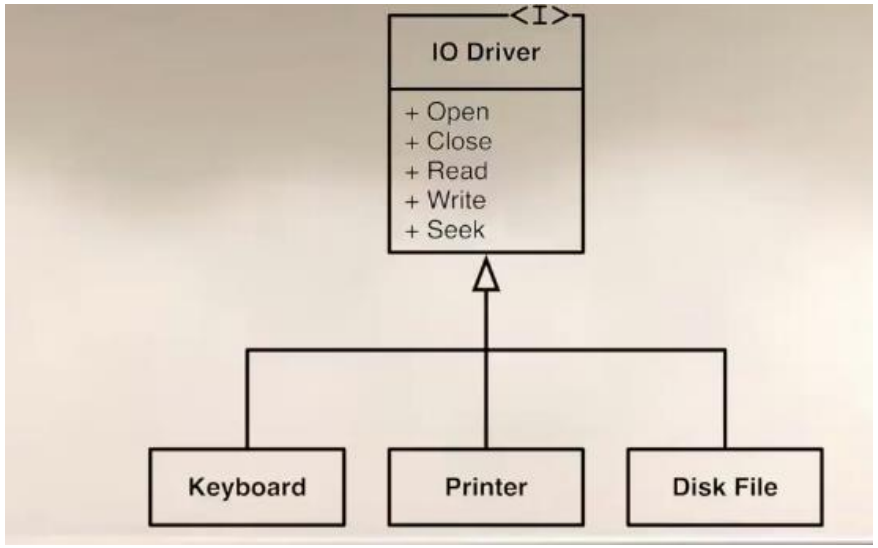
Independent Developability

- Note that all applications can be developed independently without any dependencies.

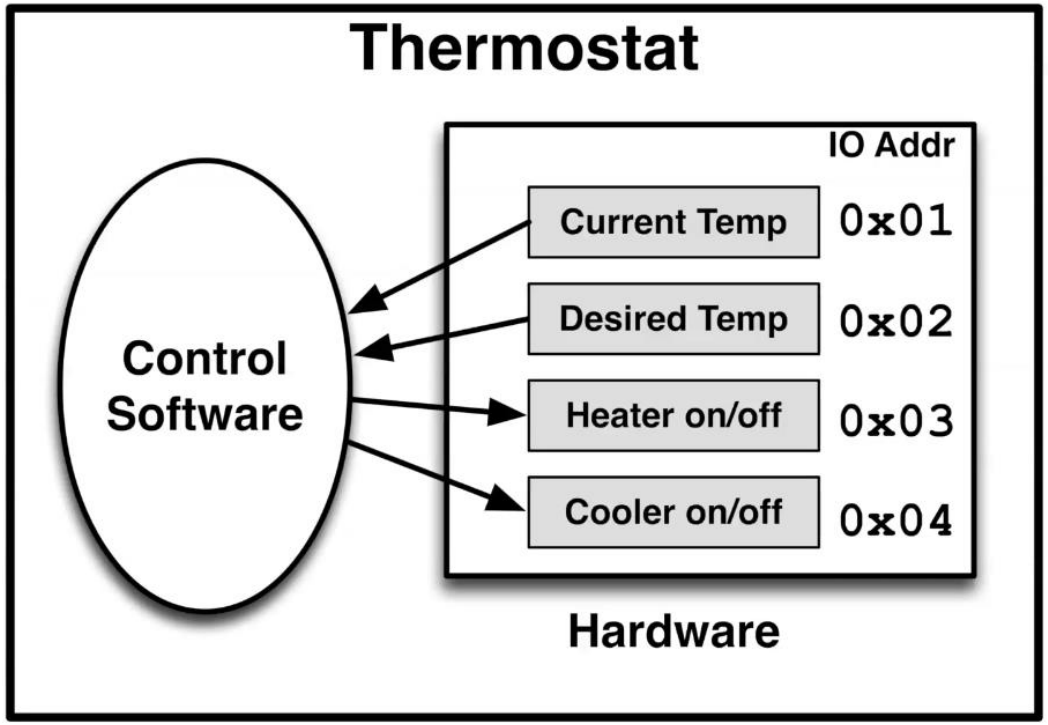
Examples

- Operating system device independence.
- putchar(‘x’)
- Must implement all of open, close, read, write, seek.
- Must also have vectors for each one.

- Device drivers are similar to plugins
  for the operating system,



Furnace Example



- This has to inputs and two outputs.

```
void regulate
  int goat_t, t;
  while(1)
  sleep(ONE_MINUTE);
  goal_t – in(0x02)
  t = in(0x01)
  if( t < goal_t )
    out(0x03,true)
    out(0x04,false)
  else if t > goal_t )
    out(0x03,false)
    out(0x04,true)
  else
   out(0x03,false)
    out(0x04,false)
```

Example code:

```
int t = in(0x02);
out(0x03, true );
```

Previous Example is not Good

- The previous example does not follow the dependency inversion principle and it will be difficult to reuse it.